A PRACTITIONER'S HANDBOOK

# Security Standards and Frameworks

A detailed breakdown of twenty methodologies, standards and frameworks — what each one is, how it is structured, and how it is brought into practice.

---

PTES · NIST SP 800-115 · OSSTMM · OWASP WSTG · OWASP API Top 10 · OWASP MASVS/MASTG · OWASP IoT Top 10 · ETSI EN 303 645 · OWASP LLM Top 10 · MITRE ATLAS · NIST AI RMF · MITRE ATT&CK Enterprise & ICS · ISA/IEC 62443 · NIST CSF 2.0 · CIS Benchmarks & Controls · CVSS v4.0 · ISO/IEC 27001 · ISO/IEC/IEEE 29119 · ISTQB

Edition: November 2026

# Contents


How to Use This Book 3

PART I · PENETRATION AND SECURITY TESTING METHODOLOGIES

1 PTES 5

2 NIST SP 800-115 8

3 OSSTMM 11

PART II · APPLICATION, API, MOBILE AND IOT SECURITY

4 OWASP WSTG 14

5 OWASP API Security Top 10 17

6 OWASP MASVS / MASTG 20

7 OWASP IoT Top 10 23

8 ETSI EN 303 645 25

PART III · ARTIFICIAL INTELLIGENCE SECURITY AND RISK

9 OWASP Top 10 for LLM Applications 28

10 MITRE ATLAS 31

11 NIST AI RMF 33

PART IV · ADVERSARY BEHAVIOUR AND OPERATIONAL TECHNOLOGY

12 MITRE ATT&CK for Enterprise 36

13 MITRE ATT&CK for ICS 39

14 ISA/IEC 62443 41

PART V · GOVERNANCE, HARDENING AND RISK RATING

15 NIST CSF 2.0 44

16 CIS Benchmarks and CIS Controls 47

17 CVSS v4.0 50

18 ISO/IEC 27001 53

PART VI · SOFTWARE TESTING FOUNDATIONS

19 ISO/IEC/IEEE 29119 56

20 ISTQB 59

PART VII · BRINGING IT ALL TOGETHER

21 Integrating the Frameworks 62

APPENDICES

A Quick Reference 66

B Glossary 68

# How to Use This Book

Security work runs on shared reference points. A client asks for a "PTES-aligned" penetration test; an auditor asks how a finding maps to ISO/IEC 27001; a SOC lead asks which ATT&CK techniques the red team used; a product team needs to know whether their smart doorbell meets ETSI EN 303 645. This book gives you a working understanding of the twenty standards and frameworks you will meet most often, and — more importantly — how practitioners actually apply them.

## Four kinds of document

The documents in this book are often lumped together as "frameworks", but they do very different jobs. Knowing which kind you are holding tells you how to use it.

| Kind | What it gives you | Examples in this book |
|---|---|---|
| **Methodology** | A process: the phases and activities of doing a test or assessment. | PTES, NIST SP 800-115, OSSTMM, OWASP WSTG, OWASP MASTG |
| **Risk list / taxonomy** | A catalogue of what goes wrong or what attackers do; used for scoping, checklists and shared vocabulary. | OWASP API, IoT and LLM Top 10s; MITRE ATT&CK (Enterprise and ICS); MITRE ATLAS |
| **Requirements standard** | Statements a system, product or organisation must meet; often assessable or certifiable. | OWASP MASVS, ETSI EN 303 645, ISA/IEC 62443, ISO/IEC 27001, CIS Benchmarks |
| **Governance / measurement framework** | A structure for managing risk, measuring severity or organising a programme. | NIST CSF 2.0, NIST AI RMF, CIS Controls, CVSS v4.0, ISO/IEC/IEEE 29119, ISTQB |

## The layered model

In practice these documents stack. A single security test usually touches every layer:

| | |
|---|---|
| **Governance** | Why we test and how results drive decisions — ISO/IEC 27001, NIST CSF 2.0, NIST AI RMF, ISA/IEC 62443-2-1 |
| **Engagement methodology** | How a test is planned, run and reported — PTES, NIST SP 800-115, OSSTMM, ISO/IEC/IEEE 29119 |
| **Technical test guides** | What to test on a given asset — WSTG, API Top 10, MASVS/MASTG, IoT Top 10, ETSI EN 303 645, LLM Top 10 |

| | |
|---|---|
| **Adversary models** | How real attackers behave — MITRE ATT&CK Enterprise, ATT&CK for ICS, ATLAS |
| **Baselines and ratings** | What "good" looks like and how bad a finding is — CIS Benchmarks/Controls, ISA/IEC 62443-3-3, CVSS v4.0 |
| **Testing discipline** | Shared vocabulary and test design — ISTQB, ISO/IEC/IEEE 29119-4 |

## How each chapter is organised

Every chapter opens with an **at-a-glance** panel (area, type, owner, whether it is mandatory, what to use it for, and the official link). It then explains **what it is**, breaks down its **structure** in detail, and spends most of its length on **how it is brought into practice** — the concrete steps, artefacts and habits practitioners use. Part VII then pulls everything together into a single worked engagement and a crosswalk you can reuse.

> **A note on versions.** Standards evolve. This book reflects the versions current at the time of writing (for example CVSS v4.0, NIST CSF 2.0, ISO/IEC 27001:2022, OWASP API Security Top 10 2023, OWASP Top 10 for LLM Applications 2025, MASVS v2). Identifiers such as test IDs and technique numbers occasionally change between releases — always cite the version you used and check the official link before relying on a specific clause in a contract or audit.

CHAPTER 1

# PTES

The Penetration Testing Execution Standard

| | |
|---|---|
| **Area** | Penetration testing |
| **Type** | Community methodology (phase model + technical guidelines) |
| **Maintained by** | A volunteer group of practitioners; launched 2009, largely stable since |
| **Mandatory?** | No. Widely cited in statements of work and RFPs as the reference pentest flow |
| **Use it for** | Overall pentest methodology and phase structure |
| **Link** | pentest-standard.org |

## What it is

PTES was created by a group of working penetration testers who were frustrated that "penetration test" meant wildly different things depending on who sold it. Some vendors delivered an automated scan with a logo on it; others delivered weeks of manual adversary simulation. PTES defines a common baseline: what a penetration test *is*, what phases it moves through, and what a client should expect to receive at the end.

It has two layers. The **main standard** describes the seven phases at a level a client or manager can understand. The **technical guidelines** go deeper, listing tools, techniques and the kinds of information to collect in each phase. The technical guidelines are now dated in places (tool names change), but the phase model has proven durable and is still the backbone of most consultancies' methodology documents.

## The seven phases

| # | Phase | Purpose | Typical outputs |
|---|---|---|---|
| 1 | Pre-engagement Interactions | Agree scope, goals, rules of engagement, legal authority, communication paths and emergency contacts. | Signed SOW, RoE, authorisation letter, scope list (IPs, URLs, apps), test windows, escalation tree. |
| 2 | Intelligence Gathering | Learn as much as possible about the target: OSINT, footprinting, infrastructure and people. | Asset inventory, domains/subdomains, employee and technology profile, exposed services. |
| 3 | Threat Modelling | Identify the assets that matter (business and data) and the threat actors and capabilities likely to target them. | Prioritised list of high-value assets and attack paths to pursue. |

| # | Phase | Purpose | Typical outputs |
|---|---|---|---|
| 4 | Vulnerability Analysis | Discover flaws through active and passive testing, then validate them to remove false positives. | Validated vulnerability list with evidence. |
| 5 | Exploitation | Bypass controls and gain access, choosing precise attacks rather than spraying exploits. | Proof of access, screenshots, compromised credentials, footholds. |
| 6 | Post Exploitation | Determine the value of the compromise: pivoting, privilege escalation, data access, persistence – within RoE. | Attack narrative showing business impact; data touched; cleanup log. |
| 7 | Reporting | Communicate results to executives and technical teams. | Executive summary, technical findings, risk ratings, remediation guidance. |

## Key ideas worth knowing

- **Scoping is a phase, not paperwork.** PTES treats pre-engagement as part of the test. A sloppy scope causes most engagement failures: testing a third party's IP you had no right to touch, missing the asset the client actually cared about, or running a destructive test during business hours.
- **Threat modelling drives effort.** PTES expects testers to spend their limited hours where real attackers would gain the most, not to test everything equally. That is what separates a pentest from a vulnerability scan.
- **Exploitation should be precise.** The standard explicitly discourages "throwing everything at the wall". Each exploitation attempt should be justified by earlier analysis, which also reduces the chance of crashing production systems.
- **Post-exploitation proves impact.** A shell on a server is a technical fact; showing that the shell leads to the payroll database is a business fact. PTES insists on the second.

## How it is brought into practice

### 1. Build your engagement lifecycle around the phases

Most consultancies map their internal workflow directly onto PTES. A typical implementation:

1. **Kick-off call and scoping questionnaire** (phase 1). Questionnaires capture number of IPs, apps, user roles, environments (prod vs. staging), cloud providers and third-party hosting. The output feeds a fixed-price quote and a Rules of Engagement document.
2. **Authorisation.** A signed letter from someone with authority over each asset. For cloud-hosted assets, check the provider's testing policy (most major providers no longer need pre-approval for common services, but prohibit things like DoS testing).
3. **Recon sprint** (phase 2) — time-boxed, often 10–15% of total effort.
4. **Threat modelling workshop** (phase 3) — even a 30-minute call with the client to ask "what would hurt most?" makes the test far more valuable.
5. **Testing** (phases 4–6), with daily status updates and immediate escalation of critical findings rather than waiting for the report.

6. **Reporting and debrief** (phase 7), followed by an optional retest window.

### 2. Write a Rules of Engagement document

The RoE is the most important artefact PTES encourages. It should state at minimum:

- In-scope and explicitly out-of-scope targets.
- Test dates and permitted hours; time zone.
- Permitted techniques (social engineering? physical? DoS? password spraying limits?).
- Data handling: what testers may view, copy or retain, and how evidence is destroyed afterwards.
- Emergency stop procedure and 24/7 contacts on both sides.
- Source IP addresses testers will use, so the blue team can distinguish them.

### 3. Pair PTES with asset-specific guides

PTES tells you *when* to do things; it does not give you a detailed checklist for a web app or a mobile app. In practice it is the outer frame, and inside phases 4–6 testers use OWASP WSTG (web), the API Security Top 10, OWASP MASTG (mobile), the OWASP IoT guides, or the LLM Top 10. Findings are then scored with CVSS and mapped to MITRE ATT&CK and control frameworks in the reporting phase.

> **Practitioner tip.** Keep a running *activity log* (timestamp, source IP, target, action) throughout phases 4–6. If something breaks in production, the log lets the client correlate quickly and proves what you did and did not do.

## Strengths and limitations

| Strengths | Limitations |
|---|---|
| • Clear, client-friendly phase model.<br>• Strong emphasis on scoping and business impact.<br>• Free and vendor-neutral. | • Technical guidelines are dated; tool lists are stale.<br>• No metrics or scoring model of its own.<br>• Not a formal standard; not auditable or certifiable. |

## Related frameworks

NIST SP 800-115 (similar phases, more formal), OSSTMM (metric-driven alternative), CVSS (scoring findings), MITRE ATT&CK (describing attack paths in the report).

CHAPTER 2

# NIST SP 800-115

Technical Guide to Information Security Testing and Assessment

| | |
|---|---|
| **Area** | Penetration testing and security assessment |
| **Type** | Government guidance (NIST Special Publication) |
| **Published by** | US National Institute of Standards and Technology, September 2008 |
| **Mandatory?** | Not mandatory in itself, but routinely referenced by US federal programmes (e.g. FedRAMP, FISMA assessments) and by contracts worldwide |
| **Use it for** | A recognised reference for planning and running security tests |
| **Link** | csrc.nist.gov/pubs/sp/800/115/final |

## What it is

SP 800-115 is NIST's guide to *planning and conducting* technical security testing and assessment, analysing the findings, and developing mitigation strategies. It is broader than penetration testing: it covers document and log reviews, scanning, wireless assessment and password cracking as well as pentests. Its value lies in being an authoritative, neutral reference that auditors, procurement teams and regulators recognise.

The publication dates from 2008, so its tool examples are old. The *process* guidance, however — how to plan, what roles exist, how to handle data and how to report — remains sound. Check the CSRC page for the current revision status before citing it in a contract.

## Structure: three families of technique

| Family | Techniques | Typical use |
|---|---|---|
| **Review techniques** (passive) | Documentation review, log review, ruleset review (firewalls, IDS), system configuration review, network sniffing, file integrity checking | Find policy and configuration gaps with no risk to systems. |
| **Target identification and analysis** | Network discovery, network port and service identification, vulnerability scanning, wireless scanning | Build an inventory and find candidate weaknesses. |
| **Target vulnerability validation** (active) | Password cracking, penetration testing, social engineering | Prove that weaknesses are exploitable and show impact. |

## The four-phase penetration testing model

Within validation, SP 800-115 describes penetration testing as four phases, with a feedback loop between Discovery and Attack:




- **Planning** — rules are identified, management approval is finalised and documented, and goals are set. No testing occurs.
- **Discovery** — information gathering and scanning, then vulnerability analysis comparing discovered services against vulnerability databases.
- **Attack** — gaining access, escalating privileges, system browsing, installing additional tools. If new information is found, the tester loops back to Discovery.
- **Reporting** — occurs in parallel with the other phases and culminates in a report describing vulnerabilities, risk ratings and mitigation guidance.

## Assessment planning and logistics

A distinctive strength of SP 800-115 is its treatment of the "boring" parts that make testing safe and defensible:

- **Developing a security assessment policy** at organisational level (who may authorise tests, how often, requirements for third parties).
- **Prioritising and scheduling** assessments based on risk and system categorisation.
- **Selecting and customising techniques** to the objective; not every assessment needs a pentest.
- **Logistics**: assessor selection and skills, location (internal/external), technical tools and resources.
- **Assessment plan** containing scope, rules, schedule, legal considerations and data handling.
- **Data handling**: collection, storage, transmission and destruction of assessment data, which is itself highly sensitive.

## How it is brought into practice

### Inside an organisation (testing programme owner)

1. **Write a security testing policy** referencing SP 800-115: frequency (e.g. annual external pentest, quarterly scans), who approves, and minimum assessor qualifications.
2. **Build an assessment calendar** prioritised by system criticality, change rate and regulatory exposure.
3. **Standardise the assessment plan template** using SP 800-115's suggested contents, so every test, internal or external, is approved the same way.
4. **Define a findings workflow**: from report, to risk register, to remediation tickets with SLAs, to retest. SP 800-115's post-testing section (root cause analysis, mitigation recommendations, reporting, remediation) is the template.

### For a testing provider

- Cite SP 800-115 in proposals as the reference methodology alongside PTES. Government and regulated clients frequently ask for it by name.
- Structure deliverables around the three families: a configuration review section (review techniques), a scan and inventory section (identification), and a validated findings section (validation).
- Adopt its data-handling guidance as your evidence retention policy (encrypt at rest, restrict access, destroy at engagement end with a certificate if requested).

> **Where it fits.** SP 800-115 is the "assessment programme" view; PTES is the "engagement" view. Many teams quote both: SP 800-115 for governance and planning, PTES for the engagement flow, and OWASP guides for the technical checklists.

## Strengths and limitations

| Strengths | Limitations |
|---|---|
| • Recognised by auditors and governments.<br>• Covers the whole assessment programme, not just pentests.<br>• Excellent on planning, legal and data-handling issues. | • Published 2008; tool references and some techniques are dated.<br>• Light on application, cloud, mobile and API testing.<br>• No severity scoring model. |

CHAPTER 3

# OSSTMM

Open Source Security Testing Methodology Manual

| | |
|---|---|
| **Area** | Operational security testing (all channels) |
| **Type** | Scientific testing methodology with its own metrics |
| **Maintained by** | ISECOM (Institute for Security and Open Methodologies); version 3 published 2010 |
| **Mandatory?** | No. Used where repeatability and measurable results are required |
| **Use it for** | Repeatable, metric-driven operational security testing |
| **Link** | isecom.org/OSSTMM.3.pdf |

## What it is

OSSTMM takes a very different philosophy from PTES. Rather than asking "can an attacker break in?", it asks "*how much* operational security does this target actually have, and can we measure it the same way every time?" It aims to produce fact-based, repeatable results that two different teams testing the same target would agree on.

OSSTMM avoids subjective risk language. Instead of saying a finding is "high risk", it measures how much of the target is exposed, which controls are in place, and which limitations (flaws) exist, and combines them into a numeric security value.

## Core concepts

### Porosity: Visibility, Access, Trust

OSSTMM views security as a separation between an asset and threats. Every point where that separation is broken is a "pore". Pores are classified as:

- **Visibility** — the target can be seen or detected (e.g. an IP that responds to probes).
- **Access** — an interaction point that allows a response (e.g. an open port, a reception desk).
- **Trust** — an interaction between targets that is accepted without authentication (e.g. one server trusting another's requests).

### The ten operational controls

| Class A – Interactive controls | Class B – Process controls |
|---|---|
| Authentication | Non-repudiation |

| Class A – Interactive controls | Class B – Process controls |
|---|---|
| Indemnification | Confidentiality |
| Resilience | Privacy |
| Subjugation | Integrity |
| Continuity | Alarm |

Class A controls directly influence interactions (e.g. authentication, or subjugation which forces interactions to occur only under the owner's conditions). Class B controls protect assets once an interaction occurs.

### Limitations

Rather than "vulnerabilities", OSSTMM classifies the flaws found into five limitation types: **Vulnerability** (a flaw that denies access or allows privileged access), **Weakness** (a flaw in a Class A control), **Concern** (a flaw in a Class B control), **Exposure** (an unjustifiable action or information that gives visibility), and **Anomaly** (an unidentified or unknown element).

### The rav and the STAR

Porosity, controls and limitations combine into the **rav** (risk assessment value), a measure of the attack surface: the amount of uncontrolled interaction. A score near 100 rav indicates a balance between porosity and controls ("perfect balance"); below means insufficient controls, above means more controls than needed. Results are recorded in the **STAR** (Security Test Audit Report), a standard form that records what was tested, how, and the quantified results.

## Channels and test types

OSSTMM covers five channels grouped into three classes: **PHYSSEC** (Human and Physical), **SPECSEC** (Wireless/spectrum) and **COMSEC** (Telecommunications and Data Networks). It also defines six test types by how much the tester and target know in advance:

| Type | Tester knows | Target knows |
|---|---|---|
| Blind | Nothing | Everything (expects the test) |
| Double Blind | Nothing | Nothing |
| Gray Box | Limited | Everything |
| Double Gray Box | Limited | Scope and time frame only |
| Tandem | Everything | Everything |
| Reversal | Everything | Nothing (tests the defenders' response) |

## The test process

OSSTMM's Four Point Process structures each test: **Induction** (learn about the target from the environment), **Inquest** (investigate emanations from the target), **Interaction** (probe the target directly) and **Intervention** (change resources the target depends on to see how it reacts). Within each channel, a set of modules runs from posture review and logistics through visibility audit, access verification, trust verification and controls verification, to alert and log review.

## How it is brought into practice

1. **Define the scope as a vector and channel.** E.g. "Data Networks channel, external vector, from the internet into the DMZ." OSSTMM requires each vector to be tested separately.
2. **Choose the test type** (blind, gray box, tandem…) and document it, because results are only comparable between tests of the same type.
3. **Run the modules systematically**, recording each pore (visibility, access, trust), each control observed, and each limitation.
4. **Calculate the rav** using ISECOM's spreadsheet or calculator.
5. **Complete the STAR** and attach it to the narrative report.
6. **Repeat after change.** Because the method is repeatable, you can retest after remediation or annually and compare rav values, which shows measurable improvement to management.

> **When to choose OSSTMM.** It shines when you need to compare security over time or across sites (e.g. 40 retail branches), when physical and human channels are in scope, or when clients want metrics instead of opinions. Many teams borrow its concepts (porosity, test types, STAR) even if they do not compute rav.

## Strengths and limitations

| Strengths | Limitations |
|---|---|
| • Repeatable and measurable.<br>• Covers human, physical, wireless and telecom as well as networks.<br>• Clear vocabulary for test types. | • Steep learning curve; dense language.<br>• rav is unfamiliar to most executives and not comparable with CVSS.<br>• Version 3 dates from 2010; v4 has been long in development. |

CHAPTER 4

# OWASP WSTG

Web Security Testing Guide

| | |
|---|---|
| **Area** | Web applications |
| **Type** | Testing methodology and test-case catalogue |
| **Maintained by** | OWASP Foundation (community project); v4.2 stable, with a rolling development version |
| **Mandatory?** | No, but the de facto reference for web application testing |
| **Use it for** | The reference methodology for web application testing |
| **Link** | owasp.org/www-project-web-security-testing-guide |

## What it is

The WSTG is a comprehensive guide to testing the security of web applications and web services. Where the OWASP Top 10 is an *awareness* document listing the most critical risk categories, the WSTG is the *how-to*: hundreds of pages describing what to test, why, and how, with tools and example payloads. Every test has a stable identifier, which makes it easy to build checklists and to reference tests in reports.

## Structure

The guide opens with a testing framework (how testing fits into the SDLC: before development, during definition and design, during development, during deployment, and during maintenance) and then lists tests in categories. Each test ID follows the pattern `WSTG-<CATEGORY>-<NN>`:

| Code | Category | Example tests |
|---|---|---|
| INFO | Information Gathering | Fingerprint web server; review webpage content for leakage; map application architecture |
| CONF | Configuration and Deployment Management | Test HTTP methods; HSTS; file extension handling; cloud storage; subdomain takeover |
| IDNT | Identity Management | Role definitions; user registration; account enumeration |
| ATHN | Authentication | Default credentials; lockout; bypass; weak password policy; MFA weaknesses |
| ATHZ | Authorization | Directory traversal; privilege escalation; insecure direct object references (IDOR) |
| SESS | Session Management | Cookie attributes; session fixation; CSRF; logout; session timeout; JWT |

| Code | Category | Example tests |
|---|---|---|
| INPV | Input Validation | Reflected/stored XSS; SQL injection; command injection; SSRF; SSTI; HTTP request smuggling |
| ERRH | Error Handling | Improper error handling; stack traces |
| CRYP | Cryptography | Weak TLS; padding oracle; sensitive data over unencrypted channels; weak encryption |
| BUSL | Business Logic | Data validation; request forgery; integrity checks; process timing; workflow circumvention; file upload |
| CLNT | Client-side | DOM XSS; JavaScript execution; HTML injection; open redirect; CORS; clickjacking; WebSockets; browser storage |
| APIT | API Testing | GraphQL testing (expanded in later development versions) |

For example, `WSTG-INPV-05` is Testing for SQL Injection, and `WSTG-ATHZ-04` is Testing for Insecure Direct Object References. Because IDs can shift between versions, always cite the version (e.g. "WSTG v4.2").

## How it is brought into practice

### 1. Build a test checklist from the WSTG

OWASP publishes a checklist spreadsheet derived from the guide. Teams typically fork it and add columns for *status* (Not started / Pass / Fail / N/A), *tester*, *evidence reference* and *notes*. Marking tests explicitly as N/A (with a reason) is as important as marking failures: it proves coverage.

### 2. Run the engagement in a sensible order

1. **INFO** and **CONF** first: map the app, its entry points, roles and technology stack. Spider and proxy every function through an intercepting proxy (e.g. Burp Suite, OWASP ZAP).
2. **IDNT, ATHN, SESS**: understand who users are and how they log in and stay logged in.
3. **ATHZ**: with at least two accounts per role, test horizontal and vertical access control on every object and function. This is where the most serious real-world findings usually live.
4. **INPV, CLNT**: injection and client-side tests across all inputs, combining automated scanning with manual payload work.
5. **BUSL**: business logic tests, which cannot be automated — skip steps in a workflow, replay discount codes, race a payment.
6. **ERRH, CRYP**: sweep for leakage and transport/crypto weaknesses.

### 3. Embed it into the SDLC

- **Requirements**: reference the OWASP ASVS (the sister "verification standard") for what the app must do; the WSTG tells you how to test it.
- **CI/CD**: automate the subset of WSTG tests that are automatable (DAST baseline scans, header checks, TLS checks) on every build.
- **Pre-release**: manual WSTG-based pentest for major releases or high-risk features.

- **Reporting**: tag each finding with its WSTG ID, a CWE, and a CVSS score so developers can look up the test and reproduce it.

> **WSTG vs. ASVS vs. Top 10.** The *Top 10* raises awareness of risk categories; *ASVS* defines verifiable security requirements at three levels; *WSTG* describes how to test. A mature programme uses all three: ASVS for requirements, WSTG for testing, Top 10 for training and executive communication.

## Strengths and limitations

| Strengths | Limitations |
|---|---|
| • Extremely comprehensive and free.<br>• Stable test IDs ideal for checklists and reports.<br>• Practical: payloads, tools and examples. | • Not a compliance standard; no pass/fail levels (use ASVS for that).<br>• Release cadence is slow; newest techniques may only be in the development version.<br>• Coverage of modern SPAs and APIs is still evolving. |

CHAPTER 5

# OWASP API Security Top 10

The most critical API-specific risks (2023 edition)

| | |
|---|---|
| **Area** | APIs (REST, GraphQL, gRPC and others) |
| **Type** | Risk awareness list |
| **Maintained by** | OWASP API Security Project; first edition 2019, current edition 2023 |
| **Mandatory?** | No. Widely used in procurement, testing scopes and API gateway product features |
| **Use it for** | Scoping and checklist for API testing |
| **Link** | owasp.org/API-Security |

## What it is

APIs are now the main attack surface of most applications. They expose business logic and data directly, are consumed by machines rather than browsers, and often lack the protective layers a web front-end provides. The API Security Top 10 exists because the classic OWASP Top 10 under-represents the risks that dominate API breaches — above all, broken authorisation.

## The 2023 list

| ID | Risk | In plain terms |
|---|---|---|
| API1 | Broken Object Level Authorization (BOLA) | Change an ID in the request ( `/orders/1001` → `/orders/1002` ) and get someone else's data. |
| API2 | Broken Authentication | Weak token handling, credential stuffing, missing brute-force protection, unsigned or poorly validated JWTs. |
| API3 | Broken Object Property Level Authorization (BOPLA) | Excessive data exposure and mass assignment: reading or writing fields you should not (e.g. setting `"isAdmin": true` ). |
| API4 | Unrestricted Resource Consumption | No limits on rate, payload size, pagination or expensive operations, leading to DoS or cost blow-ups (SMS, AI tokens). |
| API5 | Broken Function Level Authorization (BFLA) | A normal user can call admin functions, e.g. `DELETE /users/42` or `/admin/` endpoints. |
| API6 | Unrestricted Access to Sensitive Business Flows | Automating a legitimate flow at scale: ticket scalping, mass account creation, referral abuse. |
| API7 | Server Side Request Forgery (SSRF) | The API fetches a user-supplied URL, letting attackers reach internal services or cloud metadata. |

| ID | Risk | In plain terms |
|---|---|---|
| API8 | Security Misconfiguration | Verbose errors, permissive CORS, missing TLS, unnecessary HTTP methods, default configs. |
| API9 | Improper Inventory Management | Forgotten old versions ( `/v1/` ), undocumented debug endpoints, shadow APIs, unclear data flows to third parties. |
| API10 | Unsafe Consumption of APIs | Trusting data from third-party APIs too much; following their redirects; not validating their responses. |

## How it is brought into practice

### Scoping an API test

1. **Get the specification**: OpenAPI/Swagger file, GraphQL schema, Postman collection or gRPC protobufs. Without one, testing becomes partly reverse engineering (traffic capture from the mobile or web client).
2. **Inventory the endpoints** (API9): list every version, host and environment. Compare documentation to reality — undocumented endpoints are findings in their own right.
3. **Request multiple identities**: at least two users per role and one per privilege level, plus tenants if multi-tenant. BOLA/BFLA testing is impossible without them.
4. **Agree rate-limit testing boundaries** (API4/API6) so that resource-consumption tests do not cause real outages or costs.

### Turning the list into a test plan

| Risk | Core test activity |
|---|---|
| API1 BOLA | For every endpoint taking an object ID, replay User A's request with User B's token. Automate with an authorisation-matrix tool or proxy extension. |
| API2 | Test token lifetime, revocation on logout/password change, JWT algorithm confusion ( `alg: none` , key confusion), brute-force and credential-stuffing protections. |
| API3 | Compare response fields with what the UI needs; try adding undocumented properties in POST/PUT/PATCH bodies. |
| API4 / API6 | Measure rate limits per user, IP and token; send large page sizes and nested GraphQL queries; script business flows. |
| API5 | Call every admin or privileged function with low-privilege tokens; try HTTP method switching. |
| API7 | Supply internal and metadata URLs to every URL-accepting parameter (webhooks, image fetchers, importers). |
| API8 / API9 / API10 | Configuration review, endpoint discovery and version enumeration, and review of how third-party data is validated. |

### Beyond testing: building it into development

- Enforce authorisation in a central policy layer rather than per-endpoint code, and write unit tests that assert "User B cannot read User A's object".
- Generate contract tests from the OpenAPI spec and reject requests with unexpected properties (mitigates mass assignment).
- Use an API gateway for authentication, rate limiting and inventory; feed its traffic into discovery tooling to find shadow APIs.
- Track each Top 10 item as a recurring threat in your threat model template for every new API.

> **Note.** The Top 10 is a list of risk categories, not a full methodology. For detailed test cases combine it with WSTG (especially Authorization, Session and Input Validation tests and the API testing section) and with the OWASP ASVS API requirements.

CHAPTER 6

# OWASP MASVS / MASTG

Mobile Application Security Verification Standard and Testing Guide

| | |
|---|---|
| **Area** | Mobile apps (Android and iOS) |
| **Type** | Requirements standard (MASVS) + testing guide (MASTG) + weakness list (MASWE) |
| **Maintained by** | OWASP Mobile Application Security (MAS) project |
| **Mandatory?** | No, but the basis of many certification schemes, including app-store-related assessment programmes |
| **Use it for** | Mobile app security requirements and tests |
| **Link** | mas.owasp.org |

## What it is

The OWASP MAS project provides three linked resources:

- **MASVS** — the *standard*: what a secure mobile app must achieve, expressed as high-level control groups and controls.
- **MASTG** — the *testing guide*: detailed, platform-specific techniques and test cases (static and dynamic) to verify each control, plus tooling guidance and demo apps.
- **MASWE** — the *weakness enumeration*: specific weaknesses that link MASVS controls to MASTG tests, similar in spirit to CWE.

MASVS v2 (2023) simplified the earlier standard: the old L1/L2/R "levels" were moved out of the standard into **MAS testing profiles**, and the control list became more concise.

## MASVS control groups

| Group | Focus |
|---|---|
| MASVS-STORAGE | Secure storage of sensitive data on the device; preventing leakage via logs, backups, keyboard caches, screenshots. |
| MASVS-CRYPTO | Use of strong, current cryptography and correct key management (Android Keystore, iOS Keychain/Secure Enclave). |
| MASVS-AUTH | Secure authentication and authorisation, including biometric and local authentication done properly. |

| Group | Focus |
|---|---|
| MASVS-NETWORK | Secure network communication: TLS everywhere, certificate validation, pinning where justified. |
| MASVS-PLATFORM | Secure interaction with the platform and other apps: IPC, intents, deep links, WebViews, UI. |
| MASVS-CODE | Secure coding: input validation, up-to-date dependencies, enforced updates, platform version support. |
| MASVS-RESILIENCE | Resistance to reverse engineering and tampering: obfuscation, root/jailbreak detection, anti-debugging, integrity checks. |
| MASVS-PRIVACY | Privacy controls: data minimisation, transparency, user control (added in v2.1). |

## Testing profiles

- **L1** — baseline security for all apps.
- **L2** — defence in depth for apps handling sensitive data (banking, health).
- **R** — resilience against reverse engineering and tampering, for apps protecting IP or high-value logic (games, DRM, payments).
- **P** — privacy-focused tests.

## How it is brought into practice

### For development teams

1. **Select the profile** during design based on the data the app handles and its threat model (a retail app might be L1; a banking app L2+R).
2. **Turn MASVS controls into user stories / acceptance criteria**, e.g. "Auth tokens are stored only in the Keychain with `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`."
3. **Automate static checks** in CI (e.g. MobSF or semgrep rules) for obvious issues: hard-coded secrets, insecure WebView settings, cleartext traffic allowed.
4. **Schedule a MASTG-based assessment** before major releases.

### For testers: a typical MASTG workflow

1. **Set up the lab**: rooted Android device or emulator, jailbroken iOS device (or Corellium-type virtualisation), intercepting proxy, Frida/objection, jadx/Ghidra, MobSF.
2. **Static analysis**: decompile the APK/IPA; review manifest/Info.plist (exported components, ATS exceptions, permissions); search for secrets, endpoints, insecure crypto.
3. **Dynamic analysis**: install, intercept traffic (bypassing pinning if necessary to test the backend), inspect files written to the sandbox, logs and backups, exercise IPC and deep links.
4. **Resilience testing** (profile R): attempt to bypass root/jailbreak detection, hook functions, repackage the app.

5. **Backend testing**: the API behind the app is tested with the API Top 10 and WSTG — mobile clients often expose APIs never meant to be called directly.
6. **Report** each finding against its MASVS control and MASWE/MASTG test ID.

> **Common pitfall.** Treating client-side resilience as a security boundary. Obfuscation and root detection slow attackers down; they never replace server-side authorisation. MASVS-RESILIENCE is explicitly an add-on, not a substitute for the other groups.

CHAPTER 7

# OWASP IoT Top 10

The top security risks for Internet of Things devices (2018)

| | |
|---|---|
| **Area** | IoT devices and their ecosystems |
| **Type** | Risk awareness list |
| **Maintained by** | OWASP Internet of Things Project; current list 2018 |
| **Mandatory?** | No. Used for scoping assessments and as a design checklist |
| **Use it for** | Scoping and checklist for IoT device assessments |
| **Link** | owasp.org/www-project-internet-of-things |

## What it is

An IoT product is never just a device. It is an *ecosystem*: the hardware and firmware, the companion mobile app, the cloud backend and APIs, the update infrastructure, and local network protocols (Wi-Fi, BLE, Zigbee, MQTT). The OWASP IoT Top 10 highlights the ten things that most often go wrong across that ecosystem. Although the list dates from 2018, it remains an accurate description of the issues found in assessments today, and it directly influenced later regulation.

## The list

| ID | Risk | Typical example |
|---|---|---|
| I1 | Weak, Guessable, or Hardcoded Passwords | Default `admin/admin`, the same password on every unit, backdoor accounts in firmware. |
| I2 | Insecure Network Services | Telnet or debug services exposed; UPnP opening ports to the internet. |
| I3 | Insecure Ecosystem Interfaces | Weak authorisation in the cloud API or mobile app that controls the device. |
| I4 | Lack of Secure Update Mechanism | Unsigned firmware, updates over HTTP, no rollback protection, no update notifications. |
| I5 | Use of Insecure or Outdated Components | Old BusyBox, OpenSSL or Linux kernels; vulnerable third-party SDKs. |
| I6 | Insufficient Privacy Protection | Personal data collected and stored without need or consent. |
| I7 | Insecure Data Transfer and Storage | Cleartext protocols, unencrypted secrets in flash. |

| ID | Risk | Typical example |
|---|---|---|
| I8 | Lack of Device Management | No asset tracking, monitoring, decommissioning or update management at fleet level. |
| I9 | Insecure Default Settings | Insecure out-of-the-box configuration that users cannot change or harden. |
| I10 | Lack of Physical Hardening | Exposed UART/JTAG headers, readable flash chips, no secure boot. |

## How it is brought into practice

### Scoping an IoT assessment

Use the ten items as scoping questions. Each maps to a part of the ecosystem and therefore a test workstream:

- **Hardware workstream** (I10, I1, I7): teardown, identify debug ports, dump flash, extract firmware, look for secrets.
- **Firmware workstream** (I1, I4, I5, I9): unpack with binwalk or similar, review filesystem, generate an SBOM and match components to CVEs, emulate services, analyse the update process.
- **Network workstream** (I2, I7): port scan, protocol analysis, radio testing (BLE, Zigbee, sub-GHz) where in scope.
- **Ecosystem workstream** (I3, I6, I8): test mobile app (MASTG), cloud APIs (API Top 10), web portal (WSTG), and privacy flows.

### Practical test plan tips

1. Ask the vendor for **at least two units** (one may be destroyed in hardware testing) and for firmware images.
2. Test the **update path end-to-end**: can you install modified firmware? Can you downgrade to a vulnerable version?
3. Check whether **secrets are per-device or global** — a global key extracted from one unit compromises the whole fleet.
4. Map findings to the **regulatory baseline** the product must meet (ETSI EN 303 645, UK PSTI, EU Cyber Resilience Act / RED delegated act). The IoT Top 10 is not itself a compliance standard, but most items line up with mandatory provisions.

> **Going deeper.** OWASP also maintains the *IoT Security Testing Guide (ISTG)* and the *Firmware Security Testing Methodology (FSTM)*, which provide step-by-step test cases. Use the Top 10 to scope and communicate; use those guides to execute.

CHAPTER 8

# ETSI EN 303 645

Cyber Security for Consumer Internet of Things: Baseline Requirements

| | |
|---|---|
| **Area** | Consumer IoT products |
| **Type** | European Standard (EN) with mandatory and recommended provisions |
| **Published by** | ETSI Technical Committee CYBER; first EN 2020 (v2.1.1), revised since |
| **Mandatory?** | Voluntary as a standard, but forms the basis of law: the UK PSTI regime mirrors its top provisions, and it feeds EU requirements and many national labelling schemes |
| **Use it for** | Baseline security requirements for consumer IoT |
| **Link** | etsi.org/standards |

## What it is

EN 303 645 is the first globally applicable baseline for consumer IoT security. It covers products such as smart cameras, speakers, TVs, wearables, home assistants, connected appliances, door locks and baby monitors. It is designed to be **outcome-focused** (it says what must be achieved, not how) and **achievable** for manufacturers of any size. Its companion specification, **ETSI TS 103 701**, defines how to assess conformance.

## The thirteen provision groups

| Clause | Provision | What it means in practice |
|---|---|---|
| 5.1 | No universal default passwords | Passwords must be unique per device or set by the user. Brute-force protections required. |
| 5.2 | Implement a means to manage reports of vulnerabilities | Publish a vulnerability disclosure policy with a contact point and timelines. |
| 5.3 | Keep software updated | Secure, authenticated updates; publish the defined support period. |
| 5.4 | Securely store sensitive security parameters | No hard-coded critical credentials in firmware; use secure storage. |
| 5.5 | Communicate securely | Best-practice cryptography for security-relevant communications. |
| 5.6 | Minimise exposed attack surfaces | Disable unused interfaces and services; least privilege; no exposed debug. |

| Clause | Provision | What it means in practice |
|---|---|---|
| 5.7 | Ensure software integrity | Secure boot or equivalent integrity verification. |
| 5.8 | Ensure that personal data is secure | Protect personal data in transit and at rest. |
| 5.9 | Make systems resilient to outages | Keep working locally when network or power is lost; recover cleanly. |
| 5.10 | Examine system telemetry data | If telemetry is collected, check it for security anomalies. |
| 5.11 | Make it easy for users to delete user data | Simple factory reset and data deletion, including in the cloud. |
| 5.12 | Make installation and maintenance of devices easy | Secure setup with minimal steps and clear guidance. |
| 5.13 | Validate input data | Validate data from user interfaces, APIs and networks. |

Clause 6 adds data protection provisions: tell users what personal data is processed and why, obtain valid consent where needed, and let users withdraw it. Each provision is marked as **mandatory** (M), **recommended** (R), or **conditional** (applies only when a feature exists, e.g. if the device has an update mechanism).

## Regulatory connections

- **UK Product Security and Telecommunications Infrastructure (PSTI) regime** (in force since April 2024) legally requires the essence of 5.1, 5.2 and 5.3 for consumer connectable products sold in the UK. Conformance with the corresponding EN 303 645 provisions is accepted as a route to compliance.
- **EU**: the Radio Equipment Directive delegated act (with the harmonised EN 18031 series) and the Cyber Resilience Act raise the bar further; EN 303 645 is a strong foundation for both.
- **Labelling schemes** in several countries (e.g. Finland, Singapore, Germany) build on EN 303 645.

## How it is brought into practice

### For a manufacturer

1. **Gap analysis**: list every provision, decide whether it applies (conditional provisions), and record current status.
2. **Implementation conformance statement (ICS)**: TS 103 701 expects the manufacturer to declare, per provision, whether it is supported and how, and to justify any recommended provision not implemented.
3. **Engineering work**: typical changes are per-device passwords printed on labels, a signed update pipeline, secure boot, closing debug ports, and a published support period.
4. **Process work**: a vulnerability disclosure policy and a team to triage reports (5.2); a defined security update policy (5.3).
5. **Assessment**: self-assessment or third-party lab testing against TS 103 701, often leading to a certificate or label.

### For a tester

Use TS 103 701's test cases as the conformance checklist and the OWASP IoT Top 10 / ISTG as the technical attack-driven complement. Report findings with both references, e.g. "Hard-coded root password in firmware — OWASP I1; fails EN 303 645 provisions 5.1-1 and 5.4-3".

> **Tip.** Always check the current version on the ETSI portal; provision numbering is stable but wording and the set of conditional provisions have been refined across releases.

CHAPTER 9

# OWASP Top 10 for LLM Applications

The top risks for applications built on large language models (2025 edition)

| | |
|---|---|
| **Area** | AI / LLM applications, including RAG systems and agents |
| **Type** | Risk awareness list with mitigations |
| **Maintained by** | OWASP GenAI Security Project; first edition 2023, current edition 2025 |
| **Mandatory?** | No. Rapidly becoming the standard scoping document for AI application tests |
| **Use it for** | Scoping and checklist for AI / LLM testing |
| **Link** | genai.owasp.org |

## What it is

LLM applications introduce a new kind of attack surface: the model treats instructions and data as the same thing (natural language), its output is non-deterministic, and it is increasingly connected to tools, data stores and other systems. The OWASP Top 10 for LLM Applications lists the most critical risks for systems that use LLMs, covering not just the model but the surrounding application: prompts, retrieval pipelines, plugins/tools, and output handling.

## The 2025 list

| ID | Risk | In plain terms |
|---|---|---|
| LLM01 | Prompt Injection | User input (direct) or content the model reads, such as web pages, emails or documents (indirect), overrides the intended instructions. |
| LLM02 | Sensitive Information Disclosure | The model or app reveals personal data, secrets, proprietary data or other users' content. |
| LLM03 | Supply Chain | Compromised or untrustworthy models, datasets, fine-tuning adapters, or libraries. |
| LLM04 | Data and Model Poisoning | Manipulated training, fine-tuning or embedding data introduces backdoors or bias. |
| LLM05 | Improper Output Handling | Model output is passed to browsers, shells, SQL or other systems without validation (XSS, RCE, injection). |
| LLM06 | Excessive Agency | The LLM has too many tools, too much permission or too much autonomy, so a manipulated model can do real damage. |

| ID | Risk | In plain terms |
|---|---|---|
| LLM07 | System Prompt Leakage | The system prompt is extracted and reveals secrets, logic or guardrail details (system prompts should never contain secrets). |
| LLM08 | Vector and Embedding Weaknesses | RAG weaknesses: cross-tenant leakage in vector stores, poisoned documents, embedding inversion. |
| LLM09 | Misinformation | Confident but false output (hallucination) that users or systems rely on. |
| LLM10 | Unbounded Consumption | Uncontrolled inference cost or resource use: denial of wallet, model extraction via mass querying. |

## How it is brought into practice

### Scoping an LLM application test

1. **Draw the data flow**: user → app → system prompt + retrieved context → model → output → tools/downstream systems. Every arrow where untrusted text enters is a prompt injection point.
2. **List tools and permissions** the model can invoke (email, file access, code execution, database queries, payments). This defines the blast radius for LLM06.
3. **Identify trust boundaries in RAG**: who can write documents that end up in the index? Is retrieval filtered by the user's permissions? (LLM08, LLM02)
4. **Agree on cost limits** for testing (LLM10) and on whether testers may attempt jailbreaks that produce harmful content.

### Testing activities by risk

- **LLM01**: direct jailbreak and instruction-override attempts; indirect injection by planting instructions in documents, web pages, emails or tool outputs the model will read; test whether injected instructions can trigger tools.
- **LLM02/LLM07**: attempt extraction of system prompts, other users' conversation data, and data from documents the user should not see.
- **LLM05**: make the model emit HTML/JavaScript, Markdown image links (data exfiltration), SQL or shell syntax and see whether downstream components execute it.
- **LLM06**: chain injection with tool use: can a malicious document make the agent send an email or delete a file? Is there human approval for sensitive actions?
- **LLM08**: test tenant isolation in the vector store; plant poisoned documents and see if they surface for other users.
- **LLM10**: test rate limits, maximum context sizes and per-user cost caps.

### Defensive implementation

- Treat **all model output as untrusted user input**: encode, validate and sandbox it.
- Apply **least privilege** to tools; require human confirmation for high-impact actions; scope credentials per user, not per app.

- Enforce **authorisation outside the model**: filter retrieval results by the caller's permissions before they reach the prompt.
- Use layered guardrails (input/output classifiers), but never as the only control: prompt injection has no complete fix today.
- Build an **evaluation suite** of adversarial prompts mapped to Top 10 IDs and run it in CI whenever prompts, models or tools change.

> **Related.** The OWASP GenAI Security Project also publishes guidance for agentic AI systems (threats such as tool misuse, memory poisoning and identity abuse in autonomous agents). Use it alongside the Top 10 when the application is an agent rather than a chatbot. Map attack techniques to MITRE ATLAS and govern overall AI risk with the NIST AI RMF.

CHAPTER 10

# MITRE ATLAS

Adversarial Threat Landscape for Artificial-Intelligence Systems

| | |
|---|---|
| **Area** | AI / ML systems |
| **Type** | Knowledge base of adversary tactics, techniques and case studies |
| **Maintained by** | MITRE, with industry and government collaborators |
| **Mandatory?** | No. Used for threat modelling, red teaming and detection engineering |
| **Use it for** | Adversary techniques against AI/ML, for threat modelling |
| **Link** | atlas.mitre.org |

## What it is

ATLAS does for AI systems what ATT&CK does for enterprise IT: it catalogues *how* real adversaries attack AI-enabled systems, organised as tactics (the adversary's goal) and techniques (how they achieve it), backed by real-world and red-team **case studies**. It deliberately reuses ATT&CK's structure and many of its tactics so that security teams can extend existing threat models to cover AI components.

## Structure

ATLAS tactics include the familiar ATT&CK ones — Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Collection, Command and Control, Exfiltration and Impact — plus AI-specific tactics:

- **AI Model Access** — how the adversary gains access to the model: via an inference API, an AI-enabled product, physical environment access, or full model access.
- **AI Attack Staging** — preparing the attack: training proxy (surrogate) models, crafting adversarial data, backdooring models, verifying attacks offline.

Techniques have IDs prefixed `AML.T`. Examples:

| Technique | Description |
|---|---|
| LLM Prompt Injection (AML.T0051) | Crafting inputs that cause an LLM to ignore its instructions; direct and indirect variants. |
| LLM Jailbreak | Bypassing model safety controls to produce restricted output. |
| Poison Training Data (AML.T0020) | Inserting malicious samples to degrade or backdoor a model. |

| Technique | Description |
|---|---|
| Craft Adversarial Data (AML.T0043) | Inputs perturbed to cause misclassification (e.g. evading a malware or fraud classifier). |
| Exfiltration via AI Inference API (AML.T0024) | Model inversion, membership inference, or model extraction by querying the API. |
| Discover AI Model Ontology / Family | Learning what model and outputs a system uses to tailor attacks. |
| Publish Poisoned Models / Datasets | Supply-chain attacks via public model hubs and datasets. |

Technique names and IDs evolve between ATLAS releases; always reference the current matrix.

ATLAS also lists **mitigations** (e.g. limit model query rate, sanitise training data, use ensemble methods, control access to model artefacts) and **case studies** of real incidents and research demonstrations, which are valuable for convincing stakeholders that a threat is real.

## How it is brought into practice

### Threat modelling AI systems

1. **Decompose the AI system**: data sources, training pipeline, model registry, serving infrastructure, application layer, and users.
2. **Walk the ATLAS matrix** left to right for each component and ask "is this technique possible here?". Record plausible techniques with likelihood and impact.
3. **Combine with ATT&CK** for the conventional infrastructure (an attacker who compromises the training cluster via phishing is using ATT&CK techniques to reach an ATLAS goal).
4. **Select mitigations** from ATLAS and from your control frameworks (e.g. NIST AI RMF, ISO/IEC 27001 and AI-specific controls).

### AI red teaming

- Use ATLAS tactics to structure the red team plan and the report narrative, just as ATT&CK structures conventional red team reports.
- Tag every finding with ATLAS technique IDs, and with OWASP LLM Top 10 IDs where it is an LLM application flaw. ATLAS describes *adversary behaviour*; the OWASP list describes *application weaknesses*. The two are complementary.
- Re-use case studies as attack scenarios for exercises.

### Detection engineering

For deployed models, map telemetry (query logs, input distribution monitoring, abnormal query volumes) to ATLAS techniques it can detect, and identify gaps. For example, high-volume systematic querying can indicate model extraction (exfiltration via inference API).

CHAPTER 11

# NIST AI RMF

Artificial Intelligence Risk Management Framework (AI RMF 1.0)

| | |
|---|---|
| **Area** | AI governance and risk management |
| **Type** | Voluntary risk management framework with Playbook and profiles |
| **Published by** | NIST, January 2023 (NIST AI 100-1); Generative AI Profile (NIST AI 600-1) July 2024 |
| **Mandatory?** | Voluntary. Widely referenced by US agencies, regulators and enterprises; complements ISO/IEC 42001 and the EU AI Act |
| **Use it for** | Governance and risk framework for AI systems |
| **Link** | nist.gov/itl/ai-risk-management-framework |

## What it is

The AI RMF helps organisations manage the risks of designing, developing, deploying and using AI systems. It is broader than security: it covers any way an AI system can harm people, organisations or society, and frames these risks around building **trustworthy AI**. It is non-sector-specific, use-case agnostic and voluntary.

## Trustworthiness characteristics

The framework defines seven characteristics of trustworthy AI, used as the lens for identifying and measuring risk:

- **Valid and reliable** (the foundation for the others)
- **Safe**
- **Secure and resilient**
- **Accountable and transparent**
- **Explainable and interpretable**
- **Privacy-enhanced**
- **Fair, with harmful bias managed**

## The four core functions

| Function | Purpose | Example outcomes |
|---|---|---|
| **GOVERN** | Cultivate a culture of risk management; cross-cutting, informs all other functions. | AI policies and accountability; roles and training; risk tolerance defined; processes for third-party AI; workforce diversity considered. |
| **MAP** | Establish context and identify risks for a specific AI system. | Intended purpose and context documented; categorisation of the system; benefits and costs; impacts on individuals and groups identified. |
| **MEASURE** | Analyse, assess, benchmark and monitor risks and trustworthiness. | Metrics selected; system evaluated for trustworthiness (including security testing and red teaming); mechanisms for tracking risks over time. |
| **MANAGE** | Prioritise and act on risks based on projected impact. | Risk responses (mitigate, transfer, avoid, accept); third-party risks monitored; incident response and recovery; decommissioning plans. |

Each function is broken into categories and subcategories (e.g. GOVERN 1.1, MAP 2.3), and the companion **AI RMF Playbook** suggests concrete actions, documentation and references for each subcategory.

## Profiles

Profiles tailor the framework to a sector, technology or use case. The most important is the **Generative AI Profile (NIST AI 600-1)**, which identifies risks that are unique to or exacerbated by generative AI — such as confabulation, information integrity, information security (including prompt injection and data poisoning), harmful bias, data privacy, intellectual property, and value chain/component integration — and maps suggested actions to the AI RMF subcategories.

## How it is brought into practice

### Step 1: Govern – set up the programme

- Create an AI policy and an AI risk committee (or extend an existing risk committee).
- Build an **AI inventory**: every model and AI-enabled product in use or development, including third-party SaaS features.
- Define risk tolerance and approval gates (e.g. high-impact uses need committee sign-off).

### Step 2: Map – for each AI system

- Complete an AI impact assessment template: purpose, users, affected people, data sources, model provenance, deployment context, failure modes.
- Categorise the system (e.g. low/medium/high impact) to determine the depth of measurement required.

### Step 3: Measure

- Define metrics for each relevant trustworthiness characteristic: accuracy and robustness benchmarks, bias metrics, privacy tests.
- Commission **security testing and red teaming**: this is where the OWASP LLM Top 10 and MITRE ATLAS plug in as the technical methods.
- Set up production monitoring (drift, abuse, incidents).

### Step 4: Manage

- Record risks in the risk register with owners and treatment decisions.
- Implement mitigations; document residual risk acceptance.
- Integrate AI incidents into incident response; plan for model rollback and decommissioning.

> **Crosswalks.** NIST publishes crosswalks from the AI RMF to ISO/IEC 42001 (AI management systems), ISO/IEC 23894 (AI risk management) and the EU AI Act. Organisations already certified to ISO/IEC 27001 often implement AI RMF practices inside their existing ISMS and then pursue ISO/IEC 42001 certification.

CHAPTER 12

# MITRE ATT&CK for Enterprise

Adversarial Tactics, Techniques and Common Knowledge

| | |
|---|---|
| **Area** | Enterprise IT: Windows, macOS, Linux, cloud, network, containers, identity |
| **Type** | Knowledge base of real-world adversary behaviour |
| **Maintained by** | MITRE; public since 2015, updated twice a year |
| **Mandatory?** | No. The common language of threat intelligence, red teaming and detection engineering |
| **Use it for** | Threat modelling and mapping findings to detections |
| **Link** | attack.mitre.org |

## What it is

ATT&CK is a curated knowledge base of tactics and techniques that adversaries have actually used, built from public threat intelligence and incident reporting. It answers the question "*how* do attackers operate once they target us?" in a structured, shared vocabulary. Instead of saying "the attacker moved around the network", teams can say "T1021.002 — Remote Services: SMB/Windows Admin Shares", and everyone from the threat intel analyst to the SOC engineer to the auditor knows exactly what is meant.

## Structure

- **Tactics** (the "why"): the adversary's tactical goal. Enterprise ATT&CK has 14: Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, Impact.
- **Techniques and sub-techniques** (the "how"): e.g. T1566 Phishing → T1566.001 Spearphishing Attachment.
- **Procedures**: specific examples of how a named group or malware implemented a technique.
- **Groups** (threat actors, e.g. APT29), **Software** (malware and tools) and **Campaigns**.
- **Mitigations** (e.g. M1032 Multi-factor Authentication) and **detection** guidance (data sources and, in recent versions, detection strategies and analytics).

Tactic and technique sets change between releases (MITRE publishes new versions roughly every six months). Always cite the ATT&CK version you used.

## How it is brought into practice

### 1. Threat-informed defence and threat modelling

1. Identify threat groups relevant to your sector and geography (using ATT&CK Groups and your threat intelligence).
2. Load their techniques into **ATT&CK Navigator** to create a heat-map layer of "techniques used against organisations like us".
3. Overlay a second layer showing your detection and prevention coverage. The gaps are your priorities.

### 2. Detection engineering

- Tag every SIEM/EDR detection rule with the techniques it covers (Sigma rules support ATT&CK tags natively).
- Track coverage per technique as a KPI, but measure *quality* (tested, low false positive rate) rather than counting rules.
- Validate detections with **atomic tests** (e.g. Atomic Red Team) and purple-team exercises mapped to technique IDs.

### 3. Red teaming and penetration testing

- Plan adversary emulation around a chosen threat group's technique set (MITRE's adversary emulation plans and the Center for Threat-Informed Defense resources help here).
- In pentest reports, describe the attack path as a sequence of ATT&CK techniques. This lets the blue team check, technique by technique, whether they saw the activity.
- For each finding, add "detection opportunity" notes using the technique's detection guidance.

**Example: an internal pentest attack path mapped to ATT&CK**

| Step | Activity | Technique | Detected? |
|---|---|---|---|
| 1 | Phishing email with macro document | T1566.001 Spearphishing Attachment | Yes (mail gateway) |
| 2 | Macro launches PowerShell | T1059.001 PowerShell | No |
| 3 | LLMNR/NBT-NS poisoning captures hashes | T1557.001 LLMNR/NBT-NS Poisoning | No |
| 4 | Kerberoasting service account | T1558.003 Kerberoasting | No |
| 5 | Lateral movement via admin shares | T1021.002 SMB/Windows Admin Shares | Alert, not triaged |
| 6 | Dump credentials from LSASS | T1003.001 LSASS Memory | Yes (EDR blocked first attempt) |

### 4. Security architecture and control mapping

MITRE and the Center for Threat-Informed Defense publish mappings from ATT&CK techniques to NIST SP 800-53 controls, CIS Controls, cloud-native security services and more. These let you answer "which controls actually stop the techniques our adversaries use?" and justify investment in board-level language.

CHAPTER 13

# MITRE ATT&CK for ICS

Adversary tactics and techniques against industrial control systems

| | |
|---|---|
| **Area** | Operational technology (OT) and industrial control systems (ICS) |
| **Type** | Knowledge base of adversary behaviour in ICS environments |
| **Maintained by** | MITRE; launched 2020 |
| **Mandatory?** | No. Widely used by OT security vendors, asset owners and ICS-CERT style advisories |
| **Use it for** | Threat modelling for OT/ICS assessments |
| **Link** | attack.mitre.org/techniques/ics |

## What it is

Industrial environments differ fundamentally from enterprise IT. The ultimate targets are physical processes: turbines, valves, pumps, robots, chemical reactors. Attacks aim to cause unsafe conditions, damage or loss of production, not just data theft. ATT&CK for ICS captures adversary behaviour specific to these environments, based on real incidents such as attacks on Ukrainian power grids, TRITON/TRISIS against safety instrumented systems, Stuxnet, and Industroyer.

## Structure

ATT&CK for ICS has its own set of tactics, including two that have no direct enterprise equivalent:

| Tactic | Adversary goal |
|---|---|
| Initial Access | Get into the ICS environment (e.g. via engineering workstation, remote access, supply chain). |
| Execution | Run code or commands, e.g. through the controller's native interfaces. |
| Persistence | Maintain access, e.g. modify controller program or firmware. |
| Privilege Escalation | Gain higher-level permissions. |
| Evasion | Avoid detection, e.g. spoof reporting messages, rootkits on controllers. |
| Discovery | Locate PLCs, RTUs, HMIs; enumerate I/O and points. |
| Lateral Movement | Move through the ICS network, often via default credentials or program download. |
| Collection | Gather process knowledge: point and tag identification, program upload, screen capture. |

| Tactic | Adversary goal |
|---|---|
| Command and Control | Communicate with compromised systems, often over standard industrial protocols. |
| **Inhibit Response Function** | Prevent safety, protection and quality functions from responding: block alarms, disable safety systems, put devices in an unrecoverable state. |
| **Impair Process Control** | Manipulate, disable or damage physical control: modify parameters, send unauthorised command messages, spoof I/O. |
| Impact | The end effect: damage to property, loss of control, loss of view, loss of safety, loss of productivity and revenue. |

Example techniques: *Unauthorized Command Message* (T0855), *Modify Parameter* (T0836), *Modify Program* (T0889), *Device Restart/Shutdown* (T0816), *Manipulation of View* (T0832). The matrix also models **assets** (PLC, HMI, engineering workstation, safety controller, historian, RTU) so each technique can be tied to the asset types it affects.

## How it is brought into practice

### Threat modelling OT sites

1. **Build the asset inventory and network model**, typically aligned to the Purdue reference model levels (0 field devices, 1 controllers, 2 supervisory, 3 site operations, 3.5 DMZ, 4–5 enterprise).
2. **Identify crown-jewel processes** and the "consequences that matter" (e.g. a toxic release, an explosion, a city blackout), an approach known as consequence-driven engineering.
3. **Trace attack paths** using enterprise ATT&CK to reach the OT boundary and ICS ATT&CK inside it, focusing on paths that end in Inhibit Response Function and Impair Process Control.
4. **Map controls** (ISA/IEC 62443 requirements, network segmentation, monitoring) to each technique.

### Safe OT assessments

Testing in OT is very different from IT. Active scanning can crash fragile controllers. Good practice:

- Prefer **passive** techniques (traffic capture, configuration review, architecture review) on production.
- Perform active testing on **test benches, spares or digital twins**, or during planned outages.
- Involve the asset owner's operations and safety engineers in planning and during testing.
- Report findings with their ICS technique and the physical consequence they could enable.

### Detection in OT

OT network monitoring platforms map alerts (e.g. unexpected program download to a PLC, new Modbus write function codes) to ICS techniques. Asset owners use the matrix to assess monitoring coverage, just as IT SOCs do with Enterprise ATT&CK.

CHAPTER 14

# ISA/IEC 62443

Security for Industrial Automation and Control Systems

| | |
|---|---|
| **Area** | Operational technology / industrial automation and control systems (IACS) |
| **Type** | International standards series (requirements, processes, certification) |
| **Published by** | ISA99 committee and IEC TC65; adopted as IEC standards |
| **Mandatory?** | Voluntary internationally, but required by many asset owners in procurement, referenced by regulators, and the basis of ISASecure and IECEE certification |
| **Use it for** | Zones/conduits model and OT security requirements |
| **Link** | isa.org/standards-and-publications/isa-standards |

## What it is

ISA/IEC 62443 is the reference set of standards for securing industrial automation and control systems. Its key idea is that OT security is a **shared responsibility** across three roles — the *asset owner* (who runs the plant), the *system integrator / service provider* (who designs, builds and maintains the system), and the *product supplier* (who builds the components) — and it gives each role its own requirements.

## Structure of the series

| Group | Key documents | Who it is for |
|---|---|---|
| 1 – General | 62443-1-1 Concepts and models; terminology; metrics | Everyone |
| 2 – Policies and procedures | 62443-2-1 Security programme requirements for asset owners; 2-3 Patch management; 2-4 Requirements for IACS service providers | Asset owners, service providers |
| 3 – System | 62443-3-2 Security risk assessment and system design (zones and conduits); 3-3 System security requirements and security levels | Asset owners, integrators |
| 4 – Component | 62443-4-1 Secure product development lifecycle; 4-2 Technical security requirements for components | Product suppliers |

## Core concepts

### Zones and conduits

A **zone** is a grouping of assets that share the same security requirements (e.g. "Line 3 safety system", "Control room HMIs"). A **conduit** is a grouping of communication channels between zones (e.g. "historian replication from control zone to DMZ"). Security requirements are defined per zone and conduit, so you protect the boundaries and the most critical zones most strongly. This is the OT equivalent of network segmentation design, but driven by risk assessment.

### Seven foundational requirements (FRs)

1. **FR1** Identification and authentication control
2. **FR2** Use control
3. **FR3** System integrity
4. **FR4** Data confidentiality
5. **FR5** Restricted data flow
6. **FR6** Timely response to events
7. **FR7** Resource availability

62443-3-3 (systems) and 62443-4-2 (components) break each FR into detailed system requirements (SRs) / component requirements (CRs) and requirement enhancements.

### Security levels (SL)

| SL | Protection against… |
|---|---|
| SL 0 | No specific requirements |
| SL 1 | Casual or coincidental violation |
| SL 2 | Intentional violation using simple means, low resources, generic skills, low motivation |
| SL 3 | Intentional violation using sophisticated means, moderate resources, IACS-specific skills, moderate motivation |
| SL 4 | Intentional violation using sophisticated means, extended resources, IACS-specific skills, high motivation (e.g. nation states) |

Three variants are used: **SL-T** (target, what a zone needs), **SL-C** (capability, what a component or system can deliver) and **SL-A** (achieved, what is actually in place). The design goal is SL-A ≥ SL-T for every zone and conduit. 62443-4-1 additionally defines **maturity levels** for a supplier's development process.

## How it is brought into practice

### Asset owner journey (62443-2-1 and 3-2)

1. **Establish the cybersecurity management system** for OT (policy, roles, training, supplier requirements) per 62443-2-1.
2. **Inventory and define the System under Consideration (SuC)**.
3. **High-level risk assessment** to identify worst-case consequences.
4. **Partition into zones and conduits**; draw the zone and conduit diagram.
5. **Detailed risk assessment** per zone and conduit; assign an SL-T.
6. **Document cybersecurity requirements specification (CRS)** for each zone/conduit.
7. **Procure** systems and components with SL-C that meets SL-T (ask suppliers for 62443-4-1/4-2 certification and integrators for 2-4 compliance).
8. **Verify** SL-A through assessments and testing, then operate, monitor and patch (62443-2-3).

### Product supplier journey (62443-4-1 and 4-2)

- Implement a secure development lifecycle: security requirements, secure design, secure implementation, verification and validation testing, defect management, patch management, product end-of-life, and security guidelines for users.
- Design components to meet CRs at a target SL and obtain certification (e.g. ISASecure CSA/SDLA or IECEE schemes).

### Assessors and testers

OT assessments are often structured as a 62443 gap assessment: zone and conduit review, FR-by-FR evaluation of SR compliance, and SL-A estimation. Technical testing results (ideally on non-production equipment) are mapped to FRs and to ATT&CK for ICS techniques.

CHAPTER 15

# NIST CSF 2.0

The NIST Cybersecurity Framework, version 2.0

| | |
|---|---|
| **Area** | Governance and cybersecurity risk management |
| **Type** | Voluntary outcome-based framework |
| **Published by** | NIST; v1.0 2014, v1.1 2018, v2.0 February 2024 |
| **Mandatory?** | Voluntary for most organisations, but referenced by US regulators, insurers and boards worldwide |
| **Use it for** | Framing findings against Govern / Identify / Protect / Detect / Respond / Recover |
| **Link** | nist.gov/cyberframework |

## What it is

The CSF is a taxonomy of high-level cybersecurity *outcomes* that any organisation, of any size or sector, can use to understand, assess, prioritise and communicate its cybersecurity efforts. It does not tell you how to achieve outcomes; it points to other standards (informative references) for that. Version 2.0 expanded the scope beyond critical infrastructure to all organisations and added a sixth function, **Govern**, putting governance at the centre.

## The CSF Core: functions, categories, subcategories

The Core has 6 functions, 22 categories and 106 subcategories.

| Function | Categories |
|---|---|
| **GOVERN (GV)**<br>Strategy, expectations and policy are established, communicated and monitored. | GV.OC Organizational Context; GV.RM Risk Management Strategy; GV.RR Roles, Responsibilities and Authorities; GV.PO Policy; GV.OV Oversight; GV.SC Cybersecurity Supply Chain Risk Management |
| **IDENTIFY (ID)**<br>Current cybersecurity risks are understood. | ID.AM Asset Management; ID.RA Risk Assessment; ID.IM Improvement |
| **PROTECT (PR)**<br>Safeguards to manage risks are used. | PR.AA Identity Management, Authentication and Access Control; PR.AT Awareness and Training; PR.DS Data Security; PR.PS Platform Security; PR.IR Technology Infrastructure Resilience |

| Function | Categories |
|---|---|
| **DETECT (DE)**<br>Possible attacks and compromises are found and analysed. | DE.CM Continuous Monitoring; DE.AE Adverse Event Analysis |
| **RESPOND (RS)**<br>Actions regarding a detected incident are taken. | RS.MA Incident Management; RS.AN Incident Analysis; RS.CO Incident Response Reporting and Communication; RS.MI Incident Mitigation |
| **RECOVER (RC)**<br>Affected assets and operations are restored. | RC.RP Incident Recovery Plan Execution; RC.CO Incident Recovery Communication |

Subcategories are specific outcome statements, for example `PR.AA-01`: identities and credentials for authorised users, services and hardware are managed by the organisation; `ID.RA-01`: vulnerabilities in assets are identified, validated and recorded.

## Profiles and tiers

- **Organizational Profiles** describe a *Current Profile* (outcomes achieved today) and a *Target Profile* (outcomes desired). The gap between them drives an action plan. **Community Profiles** provide sector baselines.
- **Tiers** characterise the rigour of risk governance and management practices: Tier 1 Partial, Tier 2 Risk Informed, Tier 3 Repeatable, Tier 4 Adaptive. Tiers are not a maturity model to "max out"; organisations choose the tier appropriate to their risk.

NIST supports the Core with online resources: **Implementation Examples** for each subcategory, **Informative References** (mappings to SP 800-53, ISO/IEC 27001, CIS Controls and more), and **Quick-Start Guides** for small businesses, enterprise risk management and supply chain.

## How it is brought into practice

### Building a programme with CSF

1. **Scope**: whole organisation or a business unit / system.
2. **Gather information**: policies, risk registers, previous audits, pentest reports.
3. **Create the Current Profile**: assess each subcategory (e.g. on a 0–5 scale, or Not/Partially/Largely/Fully achieved) with evidence.
4. **Create the Target Profile** driven by risk appetite, regulation and threat landscape.
5. **Gap analysis and action plan**: prioritise, cost and assign owners.
6. **Implement and repeat**: re-assess annually; report progress to the board using the six functions as the storyline.

### Framing technical findings with CSF

Security testers use CSF to translate technical findings into governance language. Each finding (or root cause) is mapped to the subcategory it shows is not being achieved:

| Finding | CSF subcategory |
|---|---|
| Unpatched internet-facing server with known exploitable CVE | ID.RA-01 (vulnerabilities identified) and PR.PS-02 (software maintained) |
| No MFA on VPN | PR.AA-03 (users, services and hardware are authenticated) |
| Red team activity not detected for 5 days | DE.CM-01 (networks monitored) and DE.AE-02 (adverse events analysed) |
| Backups reachable and deletable from domain admin account | PR.DS-11 (backups created, protected, maintained and tested) |
| Unknown shadow cloud tenant discovered | ID.AM-02 (inventories of software, services and systems) |

Summed across a report, this produces a function-level heat map ("most findings are in PROTECT and DETECT") that executives can act on.

CHAPTER 16

# CIS Benchmarks and CIS Controls

Configuration baselines and prioritised security safeguards

| | |
|---|---|
| **Area** | Hardening and baseline security controls |
| **Type** | Consensus configuration benchmarks + prioritised control set |
| **Maintained by** | Center for Internet Security (CIS), a non-profit; Controls v8.1 (2024) |
| **Mandatory?** | Voluntary, but referenced by PCI DSS, cyber insurance questionnaires, regulators and many contracts as an acceptable hardening standard |
| **Use it for** | Baseline for configuration and hardening reviews |
| **Link** | cisecurity.org |

## What they are

CIS publishes two closely related but distinct resources:

- **CIS Benchmarks**: detailed, prescriptive configuration guides for specific technologies — over 100 products, including Windows, Linux distributions, macOS, cloud platforms (AWS, Azure, GCP), Kubernetes, Docker, databases, browsers, network devices and Microsoft 365. Each recommendation states the setting, rationale, audit procedure and remediation.
- **CIS Critical Security Controls** (CIS Controls): a prioritised set of 18 controls containing safeguards that defend against the most common attacks. They answer "what should we do first?".

## CIS Controls v8.1

| # | Control | # | Control |
|---|---|---|---|
| 1 | Inventory and Control of Enterprise Assets | 10 | Malware Defenses |
| 2 | Inventory and Control of Software Assets | 11 | Data Recovery |
| 3 | Data Protection | 12 | Network Infrastructure Management |
| 4 | Secure Configuration of Enterprise Assets and Software | 13 | Network Monitoring and Defense |
| 5 | Account Management | 14 | Security Awareness and Skills Training |
| 6 | Access Control Management | 15 | Service Provider Management |
| 7 | Continuous Vulnerability Management | 16 | Application Software Security |

| # | Control | # | Control |
|---|---|---|---|
| 8 | Audit Log Management | 17 | Incident Response Management |
| 9 | Email and Web Browser Protections | 18 | Penetration Testing |

The controls contain 153 safeguards, each tagged to an **Implementation Group**:

- **IG1** — "essential cyber hygiene": the minimum for every organisation (56 safeguards); typically small organisations with limited IT expertise.
- **IG2** — adds safeguards for organisations with multiple departments and sensitive data.
- **IG3** — all safeguards, for organisations facing sophisticated attacks or with significant regulatory exposure.

Version 8.1 also aligned the safeguards with NIST CSF 2.0 security functions, including a Govern function.

## Benchmark profiles

- **Level 1**: practical, prudent settings that provide clear security benefit with minimal impact on functionality.
- **Level 2**: defence in depth for high-security environments; may reduce functionality or performance.
- Some benchmarks also offer a **STIG** profile aligned with US DoD requirements, and BitLocker or Next Generation Windows Security profiles.

## How they are brought into practice

### Hardening programme with Benchmarks

1. **Select the benchmark and profile** for each platform (e.g. "CIS Microsoft Windows Server 2022 Benchmark, Level 1 — Member Server").
2. **Tailor**: test the baseline in a lab, document exceptions with business justification and compensating controls.
3. **Codify**: implement via Group Policy, Intune, Ansible, Chef, Terraform policy, or start from CIS Hardened Images in the cloud.
4. **Measure**: scan compliance using CIS-CAT Pro or benchmark-aligned policies in vulnerability scanners, cloud security posture management (CSPM) and Kubernetes tools.
5. **Monitor drift** and remediate continuously.

### Configuration reviews in security testing

In a build/configuration review, testers run benchmark audits and report the compliance percentage, the failed recommendations that matter most, and the risk of each. Good reviews do not just paste hundreds of failed checks; they group them into themes (e.g. "legacy authentication protocols enabled", "insufficient audit logging") and prioritise.

### Security programme with Controls

- Perform a CIS Controls self-assessment (CIS provides the free CIS Controls Self Assessment Tool, CSAT).
- Achieve IG1 first, then progress to IG2/IG3 based on risk.
- Use CIS mappings to other frameworks (NIST CSF, ISO/IEC 27001, PCI DSS) so one assessment serves several compliance needs.
- Use the CIS Community Defense Model, which maps safeguards to ATT&CK techniques, to show how each safeguard reduces exposure to common attack types (ransomware, web app hacking, insider misuse).

CHAPTER 17

# CVSS v4.0

Common Vulnerability Scoring System, version 4.0

| | |
|---|---|
| **Area** | Vulnerability severity rating |
| **Type** | Open scoring standard |
| **Maintained by** | FIRST (Forum of Incident Response and Security Teams); v4.0 published November 2023 |
| **Mandatory?** | De facto standard; used by NVD, CNAs and vendors for CVE scoring and by PCI DSS and many policies for remediation SLAs |
| **Use it for** | Consistent technical severity scoring of findings |
| **Link** | first.org/cvss |

## What it is

CVSS captures the principal characteristics of a vulnerability and produces a numerical score from 0.0 to 10.0 reflecting its **severity**, plus a textual **vector string** recording exactly how the score was derived. It provides a shared, reproducible language: two analysts scoring the same vulnerability with the same information should arrive at the same vector.

A key point FIRST emphasises in v4.0: **CVSS measures severity, not risk**. Risk also depends on the likelihood of exploitation and the business value of the affected asset, which the Base score alone does not capture.

## Metric groups

| Group | Metrics | Set by |
|---|---|---|
| **Base** – Exploitability | Attack Vector (AV: Network, Adjacent, Local, Physical); Attack Complexity (AC: Low, High); *Attack Requirements* (AT: None, Present) – new in v4; Privileges Required (PR: None, Low, High); User Interaction (UI: None, Passive, Active) | Vendor / analyst |
| **Base** – Impact | Vulnerable System Confidentiality, Integrity, Availability (VC/VI/VA); Subsequent System Confidentiality, Integrity, Availability (SC/SI/SA). Each High, Low or None. | Vendor / analyst |
| **Threat** | Exploit Maturity (E: Attacked, Proof-of-Concept, Unreported) | Consumer, using threat intelligence |

| Group | Metrics | Set by |
|---|---|---|
| **Environmental** | Confidentiality/Integrity/Availability Requirements (CR/IR/AR) and Modified Base metrics | Consumer (asset owner) |
| **Supplemental** | Safety (S), Automatable (AU), Recovery (R), Value Density (V), Vulnerability Response Effort (RE), Provider Urgency (U). These do *not* change the score but add context. | Vendor / consumer |

## What changed from v3.1

- The confusing **Scope** metric was removed and replaced by separate impact metrics for the vulnerable system and subsequent systems.
- **Attack Requirements** (AT) separates "conditions the attacker cannot control" (e.g. a race condition, specific deployment configuration) from Attack Complexity.
- **User Interaction** gained granularity (Passive vs. Active).
- Temporal metrics were simplified into the **Threat** group, with only Exploit Maturity.
- Scores are computed from **equivalence classes (macrovectors)** derived from expert ranking, rather than a single formula; use FIRST's calculator or a reference library.
- New nomenclature makes it explicit which groups were used: **CVSS-B** (Base), **CVSS-BT** (Base + Threat), **CVSS-BE** (Base + Environmental), **CVSS-BTE** (all).

## Qualitative severity scale

| Rating | Score |
|---|---|
| None | 0.0 |
| Low | 0.1 – 3.9 |
| Medium | 4.0 – 6.9 |
| High | 7.0 – 8.9 |
| Critical | 9.0 – 10.0 |

### Worked example: unauthenticated SQL injection in a public web app

Network-reachable (AV:N), no special conditions (AC:L, AT:N), no login (PR:N), no user interaction (UI:N), full read and write of the database and potential service disruption on the vulnerable system (VC:H/VI:H/VA:H), no direct impact on other systems (SC:N/SI:N/SA:N).

`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N` → **9.3 Critical (CVSS-B)**

If the tester also knows a public exploit is being used in the wild, add `E:A` (CVSS-BT). If the asset owner states that the database holds only public marketing content, Environmental requirements (e.g. `CR:L`) can lower the CVSS-BTE score for that organisation.

## How it is brought into practice

### In penetration test reports

- Provide the **full vector string** for every finding, not just a number — it makes the rating auditable and lets clients rescore.
- State which nomenclature you used (usually CVSS-B or CVSS-BT from a tester; the client adds Environmental).
- Score the **individual vulnerability**, not the whole attack chain. Use the narrative and ATT&CK mapping to explain chained impact.
- Some firms separate "CVSS severity" from an overall "risk rating" that adds likelihood and business context. This is good practice, but explain the method.

### In vulnerability management

- Define remediation SLAs by severity (e.g. Critical 7 days, High 30 days), but **prioritise with threat data**: combine CVSS with EPSS (Exploit Prediction Scoring System, also by FIRST), the CISA Known Exploited Vulnerabilities catalogue, and asset criticality. Frameworks such as SSVC (Stakeholder-Specific Vulnerability Categorization) provide decision trees for this.
- Re-score with Environmental metrics for your crown-jewel systems.

CHAPTER 18

# ISO/IEC 27001

Information security management systems — Requirements (2022 edition)

| | |
|---|---|
| **Area** | Governance: information security management |
| **Type** | International certifiable management-system standard |
| **Published by** | ISO/IEC JTC 1/SC 27; current edition ISO/IEC 27001:2022 (with Amendment 1:2024) |
| **Mandatory?** | Voluntary, but certification is a common contractual requirement, especially for SaaS, outsourcing and B2B suppliers |
| **Use it for** | Organisational security governance context |
| **Link** | iso.org/standard/27001 |

## What it is

ISO/IEC 27001 specifies requirements for establishing, implementing, maintaining and continually improving an **Information Security Management System (ISMS)**: a systematic, risk-based approach to managing sensitive information across people, processes and technology. Unlike most frameworks in this book, organisations can be **certified** against it by accredited certification bodies.

The standard has two parts: the **management system clauses** (4–10), which are mandatory, and **Annex A**, a reference list of information security controls that must be considered against identified risks. ISO/IEC 27002 provides detailed implementation guidance for each Annex A control.

## Clauses 4–10 (mandatory)

| Clause | Topic | Key requirements |
|---|---|---|
| 4 | Context of the organisation | Internal/external issues, interested parties and their requirements, ISMS scope. |
| 5 | Leadership | Top management commitment, information security policy, roles and responsibilities. |
| 6 | Planning | Risk assessment and risk treatment process, Statement of Applicability, security objectives, planning of changes. |
| 7 | Support | Resources, competence, awareness, communication, documented information. |
| 8 | Operation | Operational planning and control; performing risk assessments and risk treatment. |

| Clause | Topic | Key requirements |
|---|---|---|
| 9 | Performance evaluation | Monitoring and measurement, internal audit, management review. |
| 10 | Improvement | Continual improvement, nonconformity and corrective action. |

## Annex A (2022): 93 controls in four themes

| Theme | Count | Examples |
|---|---|---|
| 5 Organisational | 37 | Policies; roles; threat intelligence (5.7); asset inventory; access control; supplier security; cloud services (5.23); incident management; ICT readiness for business continuity (5.30); legal requirements; independent review (5.35) |
| 6 People | 8 | Screening; terms of employment; awareness and training; disciplinary process; remote working |
| 7 Physical | 14 | Perimeters; entry controls; physical security monitoring (7.4); equipment; clear desk; secure disposal |
| 8 Technological | 34 | Endpoint devices; privileged access; authentication; malware protection; management of technical vulnerabilities (8.8); configuration management (8.9); information deletion; data masking; data leakage prevention; backup; logging; monitoring activities (8.16); web filtering; cryptography; secure development; secure coding (8.28); security testing in development and acceptance (8.29) |

The 2022 revision introduced 11 new controls, including threat intelligence, cloud services security, ICT readiness for business continuity, physical security monitoring, configuration management, information deletion, data masking, data leakage prevention, monitoring activities, web filtering and secure coding. Controls can also be tagged with attributes (control type, security properties, cybersecurity concepts aligned to Identify/Protect/Detect/Respond/Recover, operational capabilities and security domains).

## How it is brought into practice

### Implementation roadmap

1. **Secure management commitment** and define the ISMS scope (e.g. "the SaaS platform and supporting functions in London and Bengaluru").
2. **Define the risk methodology** (asset- or scenario-based, likelihood × impact scale, risk acceptance criteria).
3. **Perform the risk assessment** and decide treatments.
4. **Produce the Statement of Applicability (SoA)**: all 93 Annex A controls, whether each is applicable, justification, and implementation status.
5. **Implement controls** and write required documented information (policy, risk process, SoA, objectives, competence evidence, operational procedures, monitoring results, audit programme, management review minutes, nonconformities).
6. **Operate** for a period (often 3+ months) to generate evidence; run internal audit and management review.

7. **Certification audit**: Stage 1 (documentation and readiness) and Stage 2 (implementation effectiveness). Then annual surveillance audits and recertification every three years.

### Where security testing fits

- **8.8 Management of technical vulnerabilities** — vulnerability scanning, pentesting and patch SLAs provide evidence.
- **8.29 Security testing in development and acceptance** — WSTG/MASTG/API testing, SAST/DAST in the pipeline.
- **5.35 Independent review of information security** and **clause 9** performance evaluation — independent pentests and assessments feed management review.
- **8.9 Configuration management** — CIS Benchmarks are a common way to define and evidence secure baselines.
- **Risk treatment** — pentest findings become risks in the register with owners, treatment plans and acceptance decisions (clause 6.1.3 and 8.3).

> **Tip for testers.** When writing reports for ISO 27001-certified clients, add an "ISO/IEC 27001:2022 Annex A reference" field to each finding. Auditors and ISMS managers can then trace findings directly to controls and nonconformities.

CHAPTER 19

# ISO/IEC/IEEE 29119

Software and systems engineering – Software testing

| | |
|---|---|
| **Area** | Software testing |
| **Type** | International standard series |
| **Published by** | ISO/IEC JTC 1/SC 7 with IEEE; first parts 2013, major revision 2021–2022 |
| **Mandatory?** | Voluntary; used in regulated and safety-related industries, government contracts and organisations formalising their test process |
| **Use it for** | Framework, processes and documentation for software testing |
| **Link** | iso.org/standard/81291.html (Part 1: General concepts) |

## What it is

ISO/IEC/IEEE 29119 is an internationally agreed set of standards for software testing that can be used within any software development lifecycle (waterfall, V-model, agile, DevOps) and by any organisation. It replaced earlier standards such as IEEE 829 (test documentation) and BS 7925 (component testing). Its aim is to give a common basis for test processes, documentation and techniques, so that testing can be planned, managed and audited consistently.

Security testing is a type of testing; 29119 provides the *process and documentation discipline* within which security tests (penetration tests, security functional tests, fuzzing) can be managed like any other test activity.

## Parts of the series

| Part | Title | Content |
|---|---|---|
| 29119-1 | General concepts | Vocabulary and concepts: test levels, types, practices, risk-based testing. |
| 29119-2 | Test processes | A three-layer process model (below). |
| 29119-3 | Test documentation | Templates and content for test documents, with examples for agile and traditional contexts. |
| 29119-4 | Test techniques | Specification-based, structure-based and experience-based techniques with coverage measures. |
| 29119-5 | Keyword-driven testing | Framework for keyword-driven test automation. |

| Part | Title | Content |
|---|---|---|
| Other parts / TRs | – | Guidance for agile projects, testing AI-based systems (TR 29119-11), test models, and others. |

## The three-layer process model (Part 2)

1. **Organisational test process** — creates and maintains organisation-wide test specifications: the *test policy* and *organisational test practices* (formerly "test strategy").
2. **Test management processes** — *test planning*, *test monitoring and control*, and *test completion*, applied to a project or to a test level/type.
3. **Dynamic test processes** — *test design and implementation*, *test environment and data management*, *test execution*, and *test incident reporting*.

The process is **risk-based**: test planning identifies product and project risks and uses them to decide what to test, how much and with which techniques.

## Key documents (Part 3)

| Organisational and management | Dynamic testing |
|---|---|
| • Test policy<br>• Organisational test practices<br>• Test plan<br>• Test status report<br>• Test completion report | • Test model specification (test design)<br>• Test case specification<br>• Test procedure specification<br>• Test data requirements / readiness report<br>• Test environment requirements / readiness report<br>• Test execution log<br>• Test incident report |

## How it is brought into practice

- **Tailoring is expected.** Organisations claim either *full* or *tailored* conformance. Agile teams map documents to lightweight artefacts: the test plan becomes a page in the wiki; test cases live in the test management tool; incident reports are tickets.
- **Security as a test type in the test plan.** A 29119-style test plan lists security testing alongside functional and performance testing, with its own risks, techniques (e.g. penetration testing per PTES/WSTG, SAST/DAST, fuzzing), entry/exit criteria, environments and responsibilities.
- **Pentest reports as test completion reports.** Mapping a pentest to 29119 gives it an auditable place in the project's evidence: scope (test plan), test cases (WSTG checklist), execution log (activity log), incidents (findings) and completion report (final report and retest).
- **Test techniques for security.** Part 4 techniques apply directly: boundary value analysis for input validation, state transition testing for authentication and session flows, decision tables for authorisation rules, and experience-based "error guessing" which underlies much of manual pentesting.

**Context.** 29119 has critics in parts of the testing community who argue it is too document-heavy for context-driven testing. The 2021–2022 revisions added much more agile guidance. Use it as a reference model to tailor, not a bureaucracy to copy.

CHAPTER 20

# ISTQB

International Software Testing Qualifications Board – body of knowledge and glossary

| | |
|---|---|
| **Area** | Software testing |
| **Type** | Certification scheme, syllabi (body of knowledge) and glossary |
| **Maintained by** | ISTQB, a not-for-profit association of national member boards; Foundation Level v4.0 (2023) |
| **Mandatory?** | No. One of the most widely held testing certifications worldwide; frequently requested in job descriptions |
| **Use it for** | Common vocabulary and test design techniques |
| **Link** | istqb.org |

## What it is

ISTQB defines a structured body of knowledge for software testing, delivered through syllabi at Foundation, Advanced and Expert levels, plus specialist modules. Its **Glossary** is a widely used reference for testing terminology. For security professionals, ISTQB matters because it supplies the shared vocabulary and systematic test design techniques that make security testing integrate cleanly with QA teams.

## The certification structure (simplified)

- **Foundation Level (CTFL)** — the core: fundamentals, testing in the SDLC, static testing, test analysis and design, test management, tools.
- **Core Advanced** — Test Analyst, Technical Test Analyst, Test Manager.
- **Specialist** — including **Security Tester (CT-SEC)**, Performance Testing, Automotive, AI Testing, Mobile Application Testing, Test Automation Engineering, and others.
- **Agile** — Agile Tester and related modules.
- **Expert** — improving the test process, test management.

## The seven testing principles

1. **Testing shows the presence, not the absence, of defects.** A clean pentest does not prove a system is secure.

2. **Exhaustive testing is impossible.** Prioritise by risk — the same logic as PTES threat modelling.
3. **Early testing saves time and money** ("shift left"): threat modelling and design review find flaws cheapest.
4. **Defects cluster together.** Where you find one injection flaw, look harder nearby.
5. **Tests wear out** (the "pesticide paradox"): rotating testers and updating test cases keeps finding new issues.
6. **Testing is context dependent.** A medical device and a marketing site need different testing.
7. **Absence-of-defects fallacy.** A defect-free system can still fail to meet user needs or be insecure by design.

## Test design techniques

| Category | Technique | Security application |
|---|---|---|
| Black-box | Equivalence partitioning | Group inputs (valid, invalid, malicious classes) so one payload represents each class. |
| | Boundary value analysis | Test lengths and numeric limits: buffer sizes, max upload sizes, integer overflows. |
| | Decision table testing | Verify authorisation rules: role × resource × action combinations. |
| | State transition testing | Authentication flows, MFA, password reset, session lifecycle, workflow bypass. |
| White-box | Statement and branch testing | Measure coverage during fuzzing; ensure error-handling branches are exercised. |
| | (Value of white-box testing) | Supports code review and SAST triage. |
| Experience-based | Error guessing | Applying knowledge of common vulnerabilities (the core of manual pentesting). |
| | Exploratory testing | Time-boxed, charter-driven security exploration of a feature. |
| | Checklist-based testing | Using WSTG, MASTG or API Top 10 checklists. |

The CTFL v4.0 syllabus also covers collaboration-based approaches (user stories, acceptance criteria, acceptance test-driven development), which is where security acceptance criteria belong.

## How it is brought into practice

- **Common language between QA and security.** Use ISTQB glossary terms (defect, failure, test basis, test oracle, test level, regression testing) in security test plans so QA teams can absorb security tests into their regression suites.
- **Training path.** CTFL gives testers fundamentals; CT-SEC adds security testing; developers and pentesters gain systematic test design skills that reduce missed cases.
- **Security regression.** After a pentest, convert each finding into an automated regression test case using the techniques above (e.g. a decision-table test that User B receives 403 on User A's resources).

- **Process alignment.** ISTQB's test process (planning, monitoring and control, analysis, design, implementation, execution, completion) aligns with ISO/IEC/IEEE 29119, so organisations often adopt both: ISTQB for skills and vocabulary, 29119 for formal process and documentation.

CHAPTER 21

# Integrating the Frameworks

One engagement, every layer – and a crosswalk you can reuse

No single framework covers a real engagement. This chapter walks through a realistic assessment and shows exactly where each standard is used, then gives a crosswalk table and a reporting template that link them.

## Scenario

A mid-sized health-tech company, certified to ISO/IEC 27001, offers a patient-facing web portal, a REST API, iOS and Android apps, a connected blood-pressure monitor, and a new LLM-powered assistant that answers questions using the patient's records (RAG). It commissions an annual security assessment.

## Walkthrough

| Stage | What happens | Frameworks used |
|---|---|---|
| 1. Programme context | The ISMS requires annual independent testing (Annex A 5.35, 8.8, 8.29). The CISO frames objectives in CSF terms: improve ID.RA and DE.CM. | ISO/IEC 27001, NIST CSF 2.0 |
| 2. Test planning | A test plan names security as a test type, defines risks, environments, entry/exit criteria and deliverables. | ISO/IEC/IEEE 29119-2/3, NIST SP 800-115 (planning, data handling) |
| 3. Pre-engagement | Scope, rules of engagement, authorisation letters, emergency contacts, test accounts for every role. | PTES phase 1 |
| 4. Threat modelling | Workshop identifies crown jewels (patient records), likely threat actors (financially motivated groups), and AI-specific threats. | PTES phase 3, MITRE ATT&CK, MITRE ATLAS, NIST AI RMF (MAP) |
| 5. Web portal | Checklist-driven manual testing. | OWASP WSTG |
| 6. API | Authorisation matrix testing across roles and tenants; rate limiting; inventory of versions. | OWASP API Security Top 10, WSTG |
| 7. Mobile apps | Static and dynamic analysis against L2 profile. | OWASP MASVS / MASTG |
| 8. Connected device | Hardware, firmware, BLE and companion-app testing; check regulatory baseline. | OWASP IoT Top 10, ETSI EN 303 645 |
| 9. LLM assistant | Direct and indirect prompt injection, RAG permission filtering, output handling, tool agency. | OWASP LLM Top 10, MITRE ATLAS |

| Stage | What happens | Frameworks used |
|---|---|---|
| 10. Infrastructure | Cloud and server configuration review; internal attack path from a compromised laptop. | CIS Benchmarks, MITRE ATT&CK |
| 11. Rating | Each finding scored with a full vector. | CVSS v4.0 (CVSS-BT) |
| 12. Reporting | Findings mapped to controls and functions; executive heat map by CSF function; detection gaps by ATT&CK technique. | PTES phase 7, NIST CSF 2.0, ISO/IEC 27001 Annex A, CIS Controls |
| 13. Remediation and retest | Findings enter the ISMS risk register; regression tests written; retest closes findings. | ISO/IEC 27001 clauses 6, 8, 10; ISTQB test techniques; 29119 completion report |

## A fully mapped finding

### Finding HT-07: Patient records accessible across accounts via API (BOLA)

| | |
|---|---|
| **Description** | The endpoint `GET /api/v2/patients/{id}/readings` returns blood-pressure readings for any patient ID when called with a valid token for a different patient. |
| **Test reference** | OWASP API Security Top 10 2023 – API1 BOLA; OWASP WSTG v4.2 – WSTG-ATHZ-04 (IDOR) |
| **Weakness** | CWE-639 Authorization Bypass Through User-Controlled Key |
| **Severity** | `CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N` → High (CVSS-B 7.1) |
| **Adversary view** | MITRE ATT&CK T1190 Exploit Public-Facing Application → Collection of patient data |
| **Control gap** | ISO/IEC 27001:2022 A.8.3 Information access restriction; A.8.26/8.28 Application security requirements / secure coding |
| **Governance view** | NIST CSF 2.0 PR.AA-05 (access permissions and authorisations managed); PR.DS-01 (data-at-rest protected) |
| **Baseline control** | CIS Controls v8.1 – Control 16 Application Software Security; Control 6 Access Control Management |
| **Remediation** | Enforce object-level authorisation in a central policy layer; add a decision-table regression test (ISTQB) asserting cross-patient access returns 403; review all endpoints taking object IDs. |

## Crosswalk: which framework answers which question?

| Question | Primary reference | Supporting |
|---|---|---|
| How do we run a pentest end-to-end? | PTES | NIST SP 800-115, OSSTMM |
| How do we run a security testing programme? | NIST SP 800-115 | ISO/IEC/IEEE 29119, ISO/IEC 27001 (8.8, 8.29) |

| Question | Primary reference | Supporting |
|---|---|---|
| How do we measure security repeatably over time? | OSSTMM | NIST CSF profiles, CIS-CAT scores |
| What do we test in a web app? | OWASP WSTG | OWASP ASVS |
| What do we test in an API? | OWASP API Security Top 10 | WSTG |
| What must a mobile app meet, and how do we verify it? | OWASP MASVS / MASTG | API Top 10 for the backend |
| What do we test in an IoT product? | OWASP IoT Top 10 | ETSI EN 303 645 (+ TS 103 701), MASTG, API Top 10 |
| Is our consumer IoT product compliant? | ETSI EN 303 645 | UK PSTI, EU RED/EN 18031, EU CRA |
| What do we test in an LLM app? | OWASP LLM Top 10 | MITRE ATLAS |
| How do we govern AI risk? | NIST AI RMF (+ AI 600-1) | ISO/IEC 42001, EU AI Act |
| How do attackers behave, and can we detect them? | MITRE ATT&CK | ATLAS (AI), ATT&CK for ICS (OT) |
| How do we secure an industrial site? | ISA/IEC 62443 | ATT&CK for ICS |
| How do we explain results to the board? | NIST CSF 2.0 | ISO/IEC 27001 |
| What is a secure configuration? | CIS Benchmarks | CIS Controls (4), ISO/IEC 27001 A.8.9 |
| How severe is this finding? | CVSS v4.0 | EPSS, CISA KEV, SSVC |
| How do we formalise testing processes and documents? | ISO/IEC/IEEE 29119 | ISTQB |
| How do we design good test cases and share vocabulary? | ISTQB | ISO/IEC/IEEE 29119-4 |
| How do we run security as a certifiable management system? | ISO/IEC 27001 | NIST CSF 2.0, CIS Controls |

## Recommended report fields

A finding template that carries the mappings above makes a single report useful to engineers, the SOC, the ISMS manager and the board at once:

- ID and title
- Affected asset(s)
- Description and evidence
- Reproduction steps
- Test reference (WSTG / MASTG / API / LLM / IoT ID)
- Weakness (CWE)
- CVSS v4.0 vector, score and nomenclature
- Business risk rating (with method)
- ATT&CK / ATLAS / ICS technique(s)
- Detection opportunity

- Control mapping (ISO 27001 Annex A, CIS Control, 62443 SR)
- CSF 2.0 subcategory
- Remediation and owner
- Retest status and date

## A maturity roadmap for adopting the frameworks

| Stage | Focus | Adopt |
|---|---|---|
| 1. Foundations | Know your assets, harden them, fix the obvious. | CIS Controls IG1, CIS Benchmarks Level 1, CVSS for prioritisation, annual PTES-style pentest. |
| 2. Structured testing | Asset-specific testing in the SDLC. | WSTG, API Top 10, MASVS/MASTG, IoT/LLM Top 10 as relevant; ISTQB skills; security in test plans. |
| 3. Threat-informed | Test and detect what real adversaries do. | MITRE ATT&CK (and ATLAS/ICS), purple teaming, detection coverage metrics. |
| 4. Governed | Board-level risk management and certification. | NIST CSF 2.0 profiles, ISO/IEC 27001 certification, NIST AI RMF, ISA/IEC 62443 for OT, ISO/IEC/IEEE 29119 for audited test processes. |
| 5. Measured and adaptive | Continuous improvement driven by metrics. | OSSTMM-style repeatable measurement, CSF Tier 3–4, CVSS-BTE with threat intelligence, regular profile reassessment. |

APPENDIX A

# Quick Reference

All twenty standards and frameworks at a glance

| Standard / Framework | Area | What it is | Use it for | Link |
|---|---|---|---|---|
| **PTES** | Penetration Testing | Penetration Testing Execution Standard: end-to-end pentest phases. | Overall pentest methodology and phase structure. | http://www.pentest-standard.org/ |
| **NIST SP 800-115** | Penetration Testing | Technical Guide to Information Security Testing and Assessment. | A recognised reference for planning and running security tests. | https://csrc.nist.gov/pubs/sp/800/115/final |
| **OSSTMM** | Security Testing | Open Source Security Testing Methodology Manual. | Repeatable, metric-driven operational security testing. | https://www.isecom.org/OSSTMM.3.pdf |
| **OWASP WSTG** | Web Apps | Web Security Testing Guide. | The reference methodology for web application testing. | https://owasp.org/www-project-web-security-testing-guide/ |
| **OWASP API Security Top 10** | APIs | Top risks specific to APIs. | Scoping and checklist for API testing. | https://owasp.org/API-Security/ |
| **OWASP MASVS / MASTG** | Mobile | Mobile App Security Verification Standard and testing guide. | Mobile app security requirements and tests. | https://mas.owasp.org/ |
| **OWASP IoT Top 10** | IoT | Top IoT security risks. | Scoping and checklist for IoT device assessments. | https://owasp.org/www-project-internet-of-things/ |
| **ETSI EN 303 645** | IoT | Cyber security for consumer IoT baseline. | Baseline security requirements for consumer IoT. | https://www.etsi.org/standards |
| **OWASP LLM Top 10** | AI | Top risks for LLM applications. | Scoping and checklist for AI / LLM testing. | https://genai.owasp.org/ |
| **MITRE ATLAS** | AI | Adversarial Threat Landscape for AI Systems. | Adversary techniques against AI/ML, for threat modelling. | https://atlas.mitre.org/ |

| Standard / Framework | Area | What it is | Use it for | Link |
|---|---|---|---|---|
| **NIST AI RMF** | AI | AI Risk Management Framework. | Governance and risk framework for AI systems. | https://www.nist.gov/itl/ai-risk-management-framework |
| **MITRE ATT&CK (Enterprise)** | IT | Adversary tactics and techniques for IT. | Threat modelling and mapping findings to detections. | https://attack.mitre.org/ |
| **MITRE ATT&CK (ICS)** | OT | Adversary techniques for industrial control systems. | Threat modelling for OT/ICS assessments. | https://attack.mitre.org/techniques/ics/ |
| **ISA/IEC 62443** | OT | Security for industrial automation and control systems. | Zones/conduits model and OT security requirements. | https://www.isa.org/standards-and-publications/isa-standards |
| **NIST CSF 2.0** | Governance | Cybersecurity Framework. | Framing findings against Govern/Identify/Protect/Detect/Respond/Recover. | https://www.nist.gov/cyberframework |
| **CIS Benchmarks / Controls** | Hardening | Configuration benchmarks and prioritised controls. | Baseline for configuration and hardening reviews. | https://www.cisecurity.org/ |
| **CVSS v4.0** | Rating | Common Vulnerability Scoring System. | Consistent technical severity scoring of findings. | https://www.first.org/cvss/ |
| **ISO/IEC 29119** | Software Testing | International software testing standard. | Framework, processes and documentation for software testing. | https://www.iso.org/standard/81291.html |
| **ISTQB** | Software Testing | Testing body of knowledge and glossary. | Common vocabulary and test design techniques. | https://www.istqb.org/ |
| **ISO/IEC 27001** | Governance | Information security management system. | Organisational security governance context. | https://www.iso.org/standard/27001 |

APPENDIX B

# Glossary

### ASVS

OWASP Application Security Verification Standard: verifiable security requirements for web apps at three levels.

### Attack path

A sequence of steps (often mapped to ATT&CK techniques) by which an attacker reaches an objective.

### BOLA / IDOR

Broken Object Level Authorization / Insecure Direct Object Reference: accessing another user's object by changing its identifier.

### Conduit

ISA/IEC 62443: a logical grouping of communication channels connecting two or more zones.

### CWE

Common Weakness Enumeration: MITRE's catalogue of software and hardware weakness types.

### EPSS

Exploit Prediction Scoring System (FIRST): the probability a vulnerability will be exploited in the next 30 days.

### Indirect prompt injection

Instructions hidden in content an LLM reads (documents, web pages, emails) that alter its behaviour.

### ISMS

Information Security Management System: the management system defined by ISO/IEC 27001.

### KEV

CISA Known Exploited Vulnerabilities catalogue.

### Porosity

OSSTMM: the openings (visibility, access, trust) between an asset and threats.

### Profile

NIST CSF / AI RMF: a description of current or target outcomes tailored to an organisation, sector or use case.

### Purdue model

Reference architecture dividing industrial networks into levels from field devices (0) to enterprise (4–5).

### RAG

Retrieval-Augmented Generation: supplying an LLM with retrieved documents as context for its answer.

### rav

OSSTMM risk assessment value: a quantitative measure of attack surface.

### Rules of Engagement (RoE)

The document defining what testers may and may not do during an engagement.

### SBOM

Software Bill of Materials: an inventory of software components in a product.

### Security Level (SL)

ISA/IEC 62443 measure (0–4) of protection against increasingly capable attackers; SL-T, SL-C, SL-A.

### SoA

Statement of Applicability: the ISO/IEC 27001 document listing Annex A controls, their applicability and status.

### SSVC

Stakeholder-Specific Vulnerability Categorization: a decision-tree method for prioritising vulnerability response.

### STAR

OSSTMM Security Test Audit Report.

### Tactic / Technique

ATT&CK/ATLAS: the adversary's goal (tactic) and the means of achieving it (technique).

### Test basis

ISTQB: the body of knowledge (requirements, specs, code) used as the basis for test analysis and design.

### Test oracle

ISTQB: a source used to determine the expected results of a test.

### Zone

ISA/IEC 62443: a grouping of logical or physical assets that share common security requirements.